==========
CODE EXPORT - AGREGACJA KODÓW
==========

📅 Data eksportu: 2025-12-13 11:23:10
📁 Katalog źródłowy: /storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651
📊 Liczba plików: 14
🔧 Rozszerzenia: .adoc, .bash, .bat, .c, .cfg, .cmd, .cpp, .css, .go, .h, .hpp, .htm, .html, .ini, .java, .js, .json, .jsx, .md, .ps1, .py, .pyi, .pyw, .pyx, .r, .rs, .rst, .sh, .sql, .swift, .toml, .ts, .tsx, .txt, .xml, .yaml, .yml

==========

📋 SPIS TREŚCI
---

1. Neuron Swarm System v7.0.py
2. README.txt
3. exporyy.py
4. marge.py
5. neuron_cell.py
6. neuron_system_v7.0.py
7. neuron_system_v7.py
8. neuron_systemv6.py
9. newfile.py
10. orchestrator.py
11. requirements.txt
12. run.bat
13. run_all.bat.txt
14. visualizer.py

==========

==========
PLIK 1/14: Neuron Swarm System v7.0.py
==========
Ścieżka: /storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651/Neuron Swarm System v7.0.py
Rozmiar: 34.2 KB
Rozszerzenie: .py
Linie: 858
Znaki: 33666

Encoding: utf-8


```
# NEURON SWARM SYSTEM v7.0 - ULEPSZONY
# Dodaje analizę semantyczną, różnorodność zdań i wizualizację

import numpy as np
import time
import random
import threading
from collections import deque, defaultdict
import json
from datetime import datetime
import os
import sys
import re
from typing import List, Dict, Tuple
import hashlib

print("=" * 70)
print("□ SYSTEM NEURONOWY v7.0 - ULEPSZONY")
print("=" * 70)

# === ULEPSZONY GENERATOR ZDAŃ ===
class UlepszonyGenerator:
    """Generuje bardziej różnorodne zdania"""

    def __init__(self):
        self.slownictwo = {
            'system': ['system', 'mechanizm', 'algorytm', 'model',
'architektura'],
            'neuron': ['neuron', 'komórka', 'jednostka', 'moduł',
'element'],
            'przetwarzanie': ['przetwarza', 'analizuje', 'przekształca',
'konwertuje', 'interpretuje'],
            'dane': ['zdania', 'wektory', 'dane', 'teksty',
'informacje'],
            'sposób': ['skutecznie', 'szybko', 'dokładnie',
'inteligentnie', 'automatycznie'],
            'cel': ['analizy', 'klasyfikacji', 'rozumienia',
'przetwarzania', 'optymalizacji']
        }

        self.szablony = [
            "{} {} {} w cel {} {}.",
            "Dla {} {} {} {} i {}.",
            "{} przy użyciu {} {} {} {}.",
            "W procesie {} {} {} poprzez {}.",
            "{} realizuje {} za pomocą {} w {}.",
            "Metoda {} {} {} dla {} {}.",
            "W {} {} jest {} przez {}.",
            "{} umożliwia {} {} w {}.",
            "Algorytm {} {} {} do {}.",
```

```
            "System {} {} {} w {}."
        ]

    def generuj_zdanie(self) -> str:
        """Generuje unikalne zdanie 8-12 słów"""
        szablon = random.choice(self.szablony)

        # Wypełnij szablon
        slowa = []
        for _ in range(szablon.count('{}')):
            kategoria = random.choice(list(self.slownictwo.keys()))
            slowa.append(random.choice(self.slownictwo[kategoria]))

        zdanie = szablon.format(*slowa)

        # Upewnij się o długości 8-12 słów
        wyrazy = zdanie.split()
        if len(wyrazy) < 8:
            dodatek = random.sample(self.slownictwo['dane'], 8 -
len(wyrazy))
            zdanie = zdanie.replace('.', ' ' + ' '.join(dodatek) + '.')
        elif len(wyrazy) > 12:
            zdanie = ' '.join(wyrazy[:12]) + '.'

        return zdanie.capitalize()

    def generuj_partie(self, liczba: int = 10) -> List[str]:
        """Generuje partię unikalnych zdań"""
        zdania = []
        for _ in range(liczba):
            zdania.append(self.generuj_zdanie())
        return zdania

# === NEURON Z ANALIZĄ SEMANTYCZNĄ ===
class NeuronSemantyczny:
    """Neuron z rzeczywistą analizą semantyczną"""

    def __init__(self, neuron_id, neuron_type):
        self.id = neuron_id
        self.type = neuron_type

        # Zaawansowany stan
        self.state = {
            "aktywnosc": random.uniform(0.3, 0.7),
            "energia": 100.0,
            "przetworzone": 0,
            "status": "gotowy",
            "doswiadczenie": 0,
            "specjalizacja": self._init_specjalizacja()
        }

        # Zaawansowana pamięć
        self.pamiec = deque(maxlen=50)
        self.slownik = defaultdict(int)
```

```
        self.wzorce = []
        self.wektory_baza = []  # Baza wektorów dla analizy podobieństw

        # Statystyki
        self.statystyki = {
            "sredni_czas": 0,
            "bledy": 0,
            "unikalne_slowa": 0,
            "wzorce_znalezione": 0
        }

    def _init_specjalizacja(self) -> Dict:
        """Inicjalizuje specjalizację neuronu"""
        specjalizacje = {
            "analiza": {"precyzja": 0.8, "szybkosc": 0.6, "glebia": 0.9},
            "pamiec": {"pojemnosc": 0.9, "trwalosc": 0.8, "dostep": 0.7},
            "logika": {"wnioskowanie": 0.9, "konsekwencja": 0.8,
"abstrakcja": 0.7},
            "kreatywnosc": {"oryginalnosc": 0.9, "elastycznosc": 0.8,
"asocjacja": 0.7}
        }
        return specjalizacje.get(self.type, {"ogolna": 0.5})

    def _analizuj_zdanie(self, zdanie: str) -> Dict:
        """Przeprowadza rzeczywistą analizę zdania"""
        slowa = re.findall(r'\b[a-zA-ZąćęłńóśźżĄĆĘŁŃÓŚŹŻ]+\b',
zdanie.lower())

        # Statystyki podstawowe
        statystyki = {
            "dlugosc": len(zdanie),
            "liczba_slow": len(slowa),
            "srednia_dlugosc_slowa": sum(len(s) for s in slowa) /
max(len(slowa), 1),
            "unikalne_slowa": len(set(slowa)),
            "znaki_specjalne": len(re.findall(r'[.,;:!?]', zdanie))
        }

        # Analiza w zależności od typu neuronu
        if self.type == "analiza":
            analiza = self._analiza_strukturalna(zdanie, slowa)
        elif self.type == "pamiec":
            analiza = self._analiza_leksykalna(zdanie, slowa)
        elif self.type == "logika":
            analiza = self._analiza_logiczna(zdanie, slowa)
        else:  # kreatywnosc
            analiza = self._analiza_kreatywna(zdanie, slowa)

        return {**statystyki, **analiza}

    def _analiza_strukturalna(self, zdanie: str, slowa: List[str]) ->
Dict:
        """Analiza struktury zdania"""
        return {
```

```
            "typ_zdania": "pytanie" if "?" in zdanie else "oznajmujące",
            "liczba_czesci": len(re.split(r'[,;:]', zdanie)),
            "poczatek": slowa[0] if slowa else "",
            "koniec": slowa[-1] if slowa else "",
            "powtorzenia": len(slowa) - len(set(slowa))
        }

    def _analiza_leksykalna(self, zdanie: str, slowa: List[str]) -> Dict:
        """Analiza słownictwa"""
        dlugie_slowa = [s for s in slowa if len(s) > 6]
        krotkie_slowa = [s for s in slowa if len(s) <= 3]

        return {
            "slowa_dlugie": len(dlugie_slowa),
            "slowa_krotkie": len(krotkie_slowa),
            "rozne_koncowki": len(set(s[-2:] for s in slowa if len(s) >
2)),
            "powtarzajace_sie": [s for s in set(slowa) if slowa.count(s)
> 1]
        }

    def _analiza_logiczna(self, zdanie: str, slowa: List[str]) -> Dict:
        """Analiza logiczna"""
        spojniki = ["i", "lub", "albo", "czy", "że", "gdy", "ponieważ"]
        zaprzeczenia = ["nie", "ani", "nigdy", "żaden"]

        return {
            "spojniki": [s for s in slowa if s in spojniki],
            "zaprzeczenia": [s for s in slowa if s in zaprzeczenia],
            "liczba_czynnikow": len([s for s in slowa if
s.endswith(('nie', 'ość', 'stwo'))]),
            "relacje": len([s for s in slowa if s in ["z", "do", "od",
"przez", "dla"]])
        }

    def _analiza_kreatywna(self, zdanie: str, slowa: List[str]) -> Dict:
        """Analiza kreatywna"""
        unikalne_kombinacje = []
        for i in range(len(slowa) - 1):
            kombinacja = f"{slowa[i]}_{slowa[i+1]}"
            unikalne_kombinacje.append(kombinacja)

        return {
            "unikalne_kombinacje": len(set(unikalne_kombinacje)),
            "metafory": len([s for s in slowa if s in ["jak", "niczym",
"podobny", "przypomina"]]),
            "nowe_skojarzenia": random.sample(slowa, min(3, len(slowa)))
if slowa else [],
            "ocena_oryginalnosci": random.uniform(0.3, 0.9)
        }

    def _generuj_wektor(self, analiza: Dict) -> List[float]:
        """Generuje semantyczny wektor na podstawie analizy"""
        wektor = np.zeros(12)  # 12-wymiarowy wektor
```

```
        # Wymiary 0-3: Podstawowe statystyki
        wektor[0] = min(analiza.get("dlugosc", 0) / 200, 1.0)
        wektor[1] = min(analiza.get("liczba_slow", 0) / 15, 1.0)
        wektor[2] = analiza.get("unikalne_slowa", 0) /
max(analiza.get("liczba_slow", 1), 1)
        wektor[3] = min(analiza.get("znaki_specjalne", 0) / 5, 1.0)

        # Wymiary 4-7: Specjalizacja neuronu
        if self.type == "analiza":
            wektor[4] = 1.0 if analiza.get("typ_zdania") == "pytanie"
else 0.5
            wektor[5] = min(analiza.get("liczba_czesci", 0) / 3, 1.0)
            wektor[6] = analiza.get("powtorzenia", 0) /
max(analiza.get("liczba_slow", 1), 1)
        elif self.type == "pamiec":
            wektor[4] = min(analiza.get("slowa_dlugie", 0) / 5, 1.0)
            wektor[5] = min(analiza.get("slowa_krotkie", 0) / 5, 1.0)
            wektor[6] = min(analiza.get("rozne_koncowki", 0) / 10, 1.0)
        elif self.type == "logika":
            wektor[4] = min(len(analiza.get("spojniki", [])) / 3, 1.0)
            wektor[5] = min(len(analiza.get("zaprzeczenia", [])) / 2,
1.0)
            wektor[6] = min(analiza.get("liczba_czynnikow", 0) / 5, 1.0)
        else:  # kreatywnosc
            wektor[4] = min(analiza.get("unikalne_kombinacje", 0) / 10,
1.0)
            wektor[5] = min(analiza.get("metafory", 0) / 2, 1.0)
            wektor[6] = analiza.get("ocena_oryginalnosci", 0.5)

        # Wymiary 8-11: Unikalność i sygnatura
        wektor[7] = random.uniform(0, 0.1)  # Szum
        wektor[8] = hash(zdanie) % 1000 / 1000 if "zdanie" in locals()
else 0.5
        wektor[9] = self.state["aktywnosc"]
        wektor[10] = self.state["doswiadczenie"] / 100
        wektor[11] = sum(wektor[:11]) / 11  # Średnia jako podsumowanie

        return wektor.tolist()

    def _znajdz_podobienstwa(self, nowy_wektor: List[float]) -> Dict:
        """Znajduje podobieństwa z istniejącymi wektorami"""
        if not self.wektory_baza:
            return {"podobne": 0, "najbardziej_podobny": None,
"srednie_podobienstwo": 0}

        podobienstwa = []
        for i, stary_wektor in enumerate(self.wektory_baza):
            # Prosta miara podobieństwa (odległość euklidesowa odwrotnie
proporcjonalna)
            dystans = np.linalg.norm(np.array(nowy_wektor) -
np.array(stary_wektor))
            podobienstwo = 1.0 / (1.0 + dystans)
            podobienstwa.append((i, podobienstwo))
```

```
        if podobienstwa:
            najbardziej_podobny = max(podobienstwa, key=lambda x: x[1])
            srednie = sum(p[1] for p in podobienstwa) / len(podobienstwa)

            return {
                "podobne": len([p for p in podobienstwa if p[1] > 0.7]),
                "najbardziej_podobny": najbardziej_podobny,
                "srednie_podobienstwo": srednie
            }

        return {"podobne": 0, "najbardziej_podobny": None,
"srednie_podobienstwo": 0}

    def przetworz_zdanie(self, zdanie: str) -> Dict:
        """Przetwarza zdanie z pełną analizą"""
        start_czas = time.time()

        # Sprawdź długość
        slowa = zdanie.split()
        if not 8 <= len(slowa) <= 12:
            return None

        try:
            # 1. Analiza semantyczna
            analiza = self._analizuj_zdanie(zdanie)

            # 2. Generuj wektor
            wektor = self._generuj_wektor(analiza)

            # 3. Znajdź podobieństwa
            podobienstwa = self._znajdz_podobienstwa(wektor)

            # 4. Zaktualizuj bazę wektorów
            self.wektory_baza.append(wektor)
            if len(self.wektory_baza) > 100:
                self.wektory_baza.pop(0)

            # 5. Aktualizuj słownik
            for slowo in re.findall(r'\b[a-zA-ZąćęłńóśźżĄĆĘŁŃÓŚŹŻ]+\b',
zdanie.lower()):
                self.slownik[slowo] += 1

            # 6. Oblicz czas
            czas_przetwarzania = time.time() - start_czas

            # 7. Przygotuj wynik
            wynik = {
                "zdanie": zdanie,
                "wektor": wektor,
                "analiza": analiza,
                "podobienstwa": podobienstwa,
                "czas": czas_przetwarzania,
                "slowa": len(slowa),
```

```
                "typ": self.type,
                "timestamp": time.time(),
                "neuron_id": self.id,
                "doswiadczenie_neuronu": self.state["doswiadczenie"]
            }

            # 8. Zaktualizuj pamięć
            self.pamiec.append(wynik)

            # 9. Aktualizuj stan neuronu
            self._aktualizuj_stan(czas_przetwarzania, analiza)

            # 10. Znajdź wzorce (co 10 przetworzonych zdań)
            if self.state["przetworzone"] % 10 == 0:
                self._znajdz_wzorce()

            return wynik

        except Exception as e:
            self.statystyki["bledy"] += 1
            print(f"[{self.id}] Błąd przetwarzania: {e}")
            return None

    def _aktualizuj_stan(self, czas: float, analiza: Dict):
        """Aktualizuje stan neuronu po przetworzeniu"""
        self.state["przetworzone"] += 1
        self.state["doswiadczenie"] = min(100,
self.state["doswiadczenie"] + 0.5)

        # Energia zależna od złożoności
        zlozonosc = analiza.get("unikalne_slowa", 0) /
max(analiza.get("liczba_slow", 1), 1)
        self.state["energia"] = max(0, self.state["energia"] - (0.5 +
zlozonosc))

        # Aktywność rośnie z doświadczeniem
        self.state["aktywnosc"] = min(1.0, self.state["aktywnosc"] +
0.01)

        # Aktualizuj statystyki
        self.statystyki["sredni_czas"] = (
            self.statystyki["sredni_czas"] * (self.state["przetworzone"]
- 1) + czas
        ) / self.state["przetworzone"]

        self.statystyki["unikalne_slowa"] = len(self.slownik)

        # Jeśli energia niska, neuron potrzebuje odpoczynku
        if self.state["energia"] < 20:
            self.state["status"] = "odpoczynek"
            self.state["energia"] = min(100, self.state["energia"] + 30)
        else:
            self.state["status"] = "gotowy"
```

```
    def _znajdz_wzorce(self):
        """Automatyczne znajdowanie wzorców w danych"""
        if len(self.wektory_baza) < 10:
            return

        # Prosta analiza klastrów (dla przykładu)
        ostatnie_wektory = self.wektory_baza[-10:]

        # Znajdź podobne grupy
        grupy = []
        for i, v1 in enumerate(ostatnie_wektory):
            grupa = [i]
            for j, v2 in enumerate(ostatnie_wektory):
                if i != j:
                    dystans = np.linalg.norm(np.array(v1) - np.array(v2))
                    if dystans < 0.5:  # Próg podobieństwa
                        grupa.append(j)

            if len(grupa) >= 3 and grupa not in grupy:
                grupy.append(grupa)

        if grupy:
            self.statystyki["wzorce_znalezione"] += len(grupy)
            self.wzorce.append({
                "timestamp": time.time(),
                "liczba_wzorców": len(grupy),
                "rozmiar_grup": [len(g) for g in grupy]
            })

    def status(self) -> Dict:
        """Zwraca szczegółowy status neuronu"""
        return {
            "id": self.id,
            "typ": self.type,
            "stan": self.state,
            "statystyki": self.statystyki,
            "pamiec": len(self.pamiec),
            "slownik": len(self.slownik),
            "wektory_w_bazie": len(self.wektory_baza),
            "wzorce": len(self.wzorce),
            "specjalizacja": self.state["specjalizacja"]
        }

# === ULEPSZONA KOMÓRKA ===
class KomorkaV7:
    """Ulepszona komórka neuronowa"""

    def __init__(self, komorka_id):
        self.id = komorka_id
        self.neurony = []
        self.kolejka = deque(maxlen=100)
        self.wyniki = []
        self.historia = []
        self.generator = UlepszonyGenerator()
```

```
        # Tworzenie 4 zaawansowanych neuronów
        typy = ["analiza", "pamiec", "logika", "kreatywnosc"]
        for i, typ in enumerate(typy):
            neuron_id = f"{komorka_id}-N{i+1}-V7"
            self.neurony.append(NeuronSemantyczny(neuron_id, typ))

        print(f"[{self.id}] Komórka v7.0 utworzona z {len(self.neurony)}
neuronami")

    def dodaj_zdania(self, zdania):
        """Dodaje zdania do kolejki"""
        dodane = 0
        for zdanie in zdania:
            slowa = zdanie.split()
            if 8 <= len(slowa) <= 12:
                self.kolejka.append(zdanie)
                dodane += 1

        self.historia.append({
            "czas": time.time(),
            "akcja": "dodano_zdania",
            "liczba": dodane,
            "kolejka": len(self.kolejka)
        })

        return dodane

    def przetworz_partie(self, rozmiar=3):
        """Przetwarza partię zdań z inteligentnym wyborem neuronu"""
        wyniki_partii = []

        for _ in range(min(rozmiar, len(self.kolejka))):
            zdanie = self.kolejka.popleft()

            # Inteligentny wybór neuronu
            if "?" in zdanie or "czy" in zdanie.lower():
                neuron = next((n for n in self.neurony if n.type ==
"logika"), self.neurony[0])
            elif len(zdanie) > 100:
                neuron = next((n for n in self.neurony if n.type ==
"analiza"), self.neurony[0])
            elif len(set(zdanie.lower().split())) > 8:
                neuron = next((n for n in self.neurony if n.type ==
"pamiec"), self.neurony[0])
            else:
                # Wybierz neuron z najwyższą energią
                neuron = max(self.neurony, key=lambda n:
n.state["energia"])

            # Przetwórz
            wynik = neuron.przetworz_zdanie(zdanie)
            if wynik:
                wynik["komorka"] = self.id
```

```
                wyniki_partii.append(wynik)
                self.wyniki.append(wynik)

        if wyniki_partii:
            self.historia.append({
                "czas": time.time(),
                "akcja": "przetworzono",
                "liczba": len(wyniki_partii),
                "kolejka": len(self.kolejka)
            })

        return wyniki_partii

    def status_komorki(self):
        """Szczegółowy status komórki"""
        statusy_neuronow = [n.status() for n in self.neurony]

        # Statystyki komórki
        lacznie_przetworzone = sum(n.state["przetworzone"] for n in
self.neurony)
        srednia_energia = np.mean([n.state["energia"] for n in
self.neurony])
        laczny_slownik = sum(len(n.slownik) for n in self.neurony)

        # Efektywność
        if len(self.historia) > 1:
            czas_pracy = self.historia[-1]["czas"] -
self.historia[0]["czas"]
            efektywnosc = lacznie_przetworzone / max(czas_pracy, 1)
        else:
            efektywnosc = 0

        return {
            "id_komorki": self.id,
            "wersja": "7.0",
            "neurony": len(self.neurony),
            "kolejka": len(self.kolejka),
            "przetworzone": len(self.wyniki),
            "statystyki": {
                "lacznie_przetworzone": lacznie_przetworzone,
                "srednia_energia": srednia_energia,
                "laczny_slownik": laczny_slownik,
                "efektywnosc": efektywnosc,
                "historia_akcji": len(self.historia)
            },
            "neurony_status": statusy_neuronow
        }

    def raport_szczegolowy(self):
        """Generuje szczegółowy raport"""
        status = self.status_komorki()

        raport = f"""
        📊 SZCZEGÓŁOWY RAPORT KOMÓRKI {self.id} v7.0
```

```
        {'='*50}

        📈 STATYSTYKI OGÓLNE:
        • Neurony: {status['neurony']}
        • W kolejce: {status['kolejka']} zdań
        • Przetworzone: {status['przetworzone']} zdań
        • Łącznie neuronów:
{status['statystyki']['lacznie_przetworzone']}
        • Średnia energia: {status['statystyki']['srednia_energia']:.1f}%
        • Słownik: {status['statystyki']['laczny_slownik']} unikalnych
słów
        • Efektywność: {status['statystyki']['efektywnosc']:.2f}
zdania/sekundę

        □ NEURONY:
        """

        for neuron_status in status["neurony_status"]:
            spec = neuron_status.get("specjalizacja", {})
            spec_str = ", ".join(f"{k}:{v:.2f}" for k, v in spec.items())

            raport += f"""
            {neuron_status['id']} [{neuron_status['typ']:12}]
              Stan: {neuron_status['stan']['status']}
              Energia: {neuron_status['stan']['energia']:6.1f}%
              Doświadczenie: {neuron_status['stan']['doswiadczenie']:.1f}
              Przetworzone: {neuron_status['stan']['przetworzone']:4d}
              Pamięć: {neuron_status['pamiec']:3d} wpisów
              Słownik: {neuron_status['slownik']:4d} słów
              Wektory: {neuron_status['wektory_w_bazie']:3d}
              Wzorce: {neuron_status['wzorce']:2d}
              Specjalizacja: {spec_str}
            """

        # Ostatnie wyniki
        if self.wyniki:
            raport += f"\n\n📝 OSTATNIE WYNIKI (3):"
            for wynik in self.wyniki[-3:]:
                podobne = wynik.get("podobienstwa", {}).get("podobne", 0)
                raport += f"""
                • {wynik['zdanie'][:60]}...
                  Neuron: {wynik['typ']} (podobne: {podobne})
                  Czas: {wynik['czas']:.3f}s, Słowa: {wynik['slowa']}
                """

        return raport

# === SYSTEM Z WIZUALIZACJĄ ===
class SystemV7:
    """System v7.0 z wizualizacją i zaawansowanymi funkcjami"""

    def __init__(self):
        self.komorki = []
        self.generator = UlepszonyGenerator()
```

```
        self.uruchomiony = False
        self.watek_przetwarzania = None
        self.historia_systemu = []

        print("\n" + "="*60)
        print("□  SYSTEM NEURONOWY v7.0 - ULEPSZONY")
        print("="*60)

    def pokaz_menu(self):
        """Główne menu systemu"""
        while True:
            print("\n" + "="*50)
            print("📋 MENU GŁÓWNE v7.0")
            print("="*50)
            print("1.  Utwórz nową komórkę (v7.0)")
            print("2.  Dodaj zdania do przetwarzania")
            print("3.  Wygeneruj i dodaj zdania")
            print("4.  Uruchom przetwarzanie w tle")
            print("5.  Zatrzymaj przetwarzanie")
            print("6.  Pokaż status wszystkich komórek")
            print("7.  Szczegółowy raport komórki")
            print("8.  Analizuj wyniki")
            print("9.  Eksportuj dane (JSON)")
            print("10. Eksportuj raport (TXT)")
            print("11. Wyczyść system")
            print("12. Test wydajności")
            print("0.  Wyjdź")
            print("-"*50)

            try:
                wybor = input("Wybierz opcję (0-12): ").strip()

                if wybor == "0":
                    self._wyjscie()
                    break
                elif wybor == "1":
                    self._opcja_1()
                elif wybor == "2":
                    self._opcja_2()
                elif wybor == "3":
                    self._opcja_3()
                elif wybor == "4":
                    self._opcja_4()
                elif wybor == "5":
                    self._opcja_5()
                elif wybor == "6":
                    self._opcja_6()
                elif wybor == "7":
                    self._opcja_7()
                elif wybor == "8":
                    self._opcja_8()
                elif wybor == "9":
                    self._opcja_9()
                elif wybor == "10":
```

```
                self._opcja_10()
            elif wybor == "11":
                self._opcja_11()
            elif wybor == "12":
                self._opcja_12()
            else:
                print("❌ Nieprawidłowy wybór")

        except KeyboardInterrupt:
            print("\n\n⏸  Przerwano")
            self._wyjscie()
            break
        except Exception as e:
            print(f"❌ Błąd: {e}")

    # OPCJE (podobne ale ulepszone) - pokażę tylko nowe opcje 8, 10, 11,
12

    def _opcja_8(self):
        """8. Analizuj wyniki"""
        if not self.komorki or all(len(k.wyniki) == 0 for k in
self.komorki):
            print("❌ Brak wyników do analizy")
            return

        print("\n" + "="*50)
        print("📊 ANALIZA WYNIKÓW")
        print("="*50)

        # Zbierz wszystkie wyniki
        wszystkie_wyniki = []
        for komorka in self.komorki:
            wszystkie_wyniki.extend(komorka.wyniki)

        if not wszystkie_wyniki:
            print("⚠️  Brak danych do analizy")
            return

        # Podstawowe statystyki
        print(f"\n📈 STATYSTYKI OGÓLNE:")
        print(f"• Łączna liczba wyników: {len(wszystkie_wyniki)}")
        print(f"• Liczba komórek: {len(self.komorki)}")

        # Średni czas przetwarzania
        czasy = [w.get("czas", 0) for w in wszystkie_wyniki]
        sredni_czas = sum(czasy) / len(czasy)
        print(f"• Średni czas przetwarzania: {sredni_czas:.3f}s")
        print(f"• Najkrótszy czas: {min(czasy):.3f}s")
        print(f"• Najdłuższy czas: {max(czasy):.3f}s")

        # Rozkład typów neuronów
        typy_neuronow = [w.get("typ", "nieznany") for w in
wszystkie_wyniki]
```

```
        licznik_typow = {}
        for typ in typy_neuronow:
            licznik_typow[typ] = licznik_typow.get(typ, 0) + 1

        print(f"\n□ ROZKŁAD TYPÓW NEURONÓW:")
        for typ, liczba in licznik_typow.items():
            procent = (liczba / len(wszystkie_wyniki)) * 100
            print(f"• {typ:12}: {liczba:3d} ({procent:.1f}%)")

        # Analiza długości zdań
        dlugosci = [w.get("slowa", 0) for w in wszystkie_wyniki]
        print(f"\n📏 DŁUGOŚĆ ZDAŃ:")
        print(f"• Średnia długość: {np.mean(dlugosci):.1f} słów")
        print(f"• Zakres: {min(dlugosci)}-{max(dlugosci)} słów")
        print(f"• Mediana: {np.median(dlugosci):.1f} słów")

        # Unikalne słowa
        wszystkie_slowa = set()
        for wynik in wszystkie_wyniki:
            zdanie = wynik.get("zdanie", "")
            slowa = re.findall(r'\b[a-zA-Zącęłńóśźż]+\b',
zdanie.lower())
            wszystkie_slowa.update(slowa)

        print(f"\n📚 SŁOWNICTWO:")
        print(f"• Unikalnych słów: {len(wszystkie_slowa)}")
        print(f"• Przykładowe słowa: {',
'.join(list(wszystkie_slowa)[:10])}")

        # Analiza podobieństw
        if len(wszystkie_wyniki) > 1:
            print(f"\n🔗 ANALIZA PODOBIEŃSTW:")
            liczba_podobnych = sum(
                1 for w in wszystkie_wyniki
                if w.get("podobienstwa", {}).get("podobne", 0) > 0
            )
            print(f"• Zdań z podobieństwami: {liczba_podobnych}")
            print(f"• Procent:
{(liczba_podobnych/len(wszystkie_wyniki))*100:.1f}%")

        # Sugestie
        print(f"\n💡 SUGESTIE:")
        if sredni_czas > 0.1:
            print("• Czas przetwarzania jest wysoki - rozważ
optymalizację")
        if max(czasy) > 0.2:
            print("• Niektóre zdania przetwarzają się bardzo długo")
        if len(wszystkie_slowa) < 50:
            print("• Słownik jest mały - dodaj bardziej różnorodne
zdania")

    def _opcja_10(self):
        """10. Eksportuj raport (TXT)"""
```

```
        if not self.komorki:
            print("✗ Brak komórek do raportowania")
            return

        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        nazwa_pliku = f"neuron_raport_{timestamp}.txt"

        with open(nazwa_pliku, 'w', encoding='utf-8') as f:
            f.write("="*70 + "\n")
            f.write("RAPORT SYSTEMU NEURONOWEGO v7.0\n")
            f.write("="*70 + "\n\n")
            f.write(f"Data generowania: {datetime.now().strftime('%Y-%m-
%d %H:%M:%S')}\n")
            f.write(f"Liczba komórek: {len(self.komorki)}\n")
            f.write(f"Łączna liczba neuronów: {len(self.komorki) *
4}\n\n")

            laczne_wyniki = sum(len(k.wyniki) for k in self.komorki)
            f.write(f"Łącznie przetworzonych zdań: {laczne_wyniki}\n")

            for komorka in self.komorki:
                f.write("\n" + "="*60 + "\n")
                f.write(f"KOMÓRKA: {komorka.id}\n")
                f.write("="*60 + "\n")

                status = komorka.status_komorki()
                f.write(f"\nStatystyki:\n")
                f.write(f"• Neurony: {status['neurony']}\n")
                f.write(f"• W kolejce: {status['kolejka']}\n")
                f.write(f"• Przetworzone: {status['przetworzone']}\n")
                f.write(f"• Średnia energia:
{status['statystyki']['srednia_energia']:.1f}%\n")
                f.write(f"• Słownik:
{status['statystyki']['laczny_slownik']} słów\n")

                f.write(f"\nNeurony:\n")
                for neuron_status in status['neurony_status']:
                    f.write(f"• {neuron_status['id']}: ")
                    f.write(f"E={neuron_status['stan']['energia']:.1f}%,
")
                    f.write(f"P={neuron_status['stan']['przetworzone']},
")
                    f.write(f"S={neuron_status['slownik']} słów\n")

                if komorka.wyniki:
                    f.write(f"\nOstatnie wyniki:\n")
                    for wynik in komorka.wyniki[-3:]:
                        f.write(f"• {wynik['zdanie'][:50]}...\n")
                        f.write(f"  [{wynik['typ']}]
{wynik['czas']:.3f}s, {wynik['slowa']} słów\n")

            f.write("\n" + "="*70 + "\n")
            f.write("KONIEC RAPORTU\n")
            f.write("="*70 + "\n")
```

```
        print(f"✅ Raport zapisany do: {nazwa_pliku}")
        print(f"   • Rozmiar: {os.path.getsize(nazwa_pliku) / 1024:.1f}
KB")

    def _opcja_11(self):
        """11. Wyczyść system"""
        if not self.komorki:
            print("ℹ  System jest już pusty")
            return

        potwierdzenie = input("Czy na pewno wyczyścić CAŁY system? (t/n):
").strip().lower()

        if potwierdzenie == 't':
            # Zatrzymaj przetwarzanie jeśli działa
            if self.uruchomiony:
                self.uruchomiony = False
                if self.watek_przetwarzania:
                    self.watek_przetwarzania.join(timeout=1)

            # Podsumowanie przed czyszczeniem
            laczne_wyniki = sum(len(k.wyniki) for k in self.komorki)
            laczny_slownik = sum(
                sum(len(n.slownik) for n in k.neurony)
                for k in self.komorki
            )

            # Wyczyść
            self.komorki.clear()
            self.historia_systemu.append({
                "czas": time.time(),
                "akcja": "system_wyczyszczony",
                "wyniki_usuniete": laczne_wyniki,
                "slownik_usuniety": laczny_slownik
            })

            print(f"✅ System wyczyszczony")
            print(f"   • Usunięte komórki: {len(self.komorki)}")
            print(f"   • Utracone wyniki: {laczne_wyniki}")
            print(f"   • Utracony słownik: {laczny_slownik} słów")
        else:
            print("❌ Anulowano czyszczenie")

    def _opcja_12(self):
        """12. Test wydajności"""
        print("\n" + "="*50)
        print("⚡ TEST WYDAJNOŚCI SYSTEMU")
        print("="*50)

        # Utwórz tymczasową komórkę do testu
        test_komorka = KomorkaV7("TEST-WYDAJNOSCI")
```

```
        # Wygeneruj testowe zdania
        print("\n🔧 Przygotowanie testu...")
        test_zdania = self.generator.generuj_partie(20)
        test_komorka.dodaj_zdania(test_zdania)

        # Test 1: Przetwarzanie pojedynczych zdań
        print("\n📊 TEST 1: Przetwarzanie pojedyncze")
        czasy = []
        for zdanie in test_zdania[:5]:
            start = time.time()
            wynik = test_komorka.neurony[0].przetworz_zdanie(zdanie)
            czas = time.time() - start
            czasy.append(czas)
            print(f"   • {zdanie[:40]}...: {czas:.3f}s")

        srednia_poj = sum(czasy) / len(czasy)
        print(f"   ŚREDNIA: {srednia_poj:.3f}s na zdanie")

        # Test 2: Przetwarzanie partii
        print("\n📊 TEST 2: Przetwarzanie partii (5 zdań)")
        start = time.time()
        wyniki = []
        for _ in range(5):
            if test_komorka.kolejka:
                wynik = test_komorka.przetworz_partie(1)
                if wynik:
                    wyniki.extend(wynik)
        czas_partii = time.time() - start

        print(f"   • Czas całkowity: {czas_partii:.3f}s")
        print(f"   • Średnia: {czas_partii/len(wyniki):.3f}s na zdanie")

        # Test 3: Współbieżność (symulowana)
        print("\n📊 TEST 3: Symulacja współbieżności")
        if len(test_komorka.neurony) >= 4:
            czasy_neuronow = []
            for i, neuron in enumerate(test_komorka.neurony[:4]):
                start = time.time()
                neuron.przetworz_zdanie(test_zdania[i])
                czas = time.time() - start
                czasy_neuronow.append(czas)
                print(f"   • Neuron {neuron.type}: {czas:.3f}s")

            print(f"   • Różnica najszybszy-najwolniejszy:
{max(czasy_neuronow)-min(czasy_neuronow):.3f}s")

        # Podsumowanie
        print("\n" + "="*50)
        print("📈 PODSUMOWANIE WYDAJNOŚCI:")
        print(f"• Pojedyncze zdanie: {srednia_poj:.3f}s")
        print(f"• Partia (5 zdań): {czas_partii:.3f}s
({czas_partii/5:.3f}s na zdanie)")
        print(f"• Zalecana wielkość partii: 3-5 zdań")
```

```
        # Usuń tymczasową komórkę
        del test_komorka
        print("\n✅ Test zakończony")

# === URUCHOMIENIE ===
if __name__ == "__main__":
    print("⚡ Ładowanie Systemu Neuronowego v7.0...")
    time.sleep(0.5)

    system = SystemV7()
    system.pokaz_menu()
```

==========
KONIEC PLIKU: Neuron Swarm System v7.0.py
==========

==========
PLIK 2/14: README.txt
==========
Ścieżka: /storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651/README.txt
Rozmiar: 1.2 KB
Rozszerzenie: .txt
Linie: 42
Znaki: 1092
Encoding: utf-8

```
NEURON SWARM SYSTEM v7.0
========================

📋 OPIS SYSTEMU:
System neuronowy przetwarzający zdania w wektory semantyczne.
3 komórki × 4 neurony = 12 neuronów przetwarzających równolegle.

🚀 JAK URUCHOMIĆ:
1. Kliknij dwukrotnie: run.bat
2. Lub w konsoli: python neuron_system.py

📋 MENU SYSTEMU:
1. Utwórz komórkę neuronową
2. Dodaj zdania do przetwarzania
3. Uruchom przetwarzanie w tle
4. Zatrzymaj przetwarzanie
5. Pokaż status wszystkich komórek
6. Pokaż szczegółowy raport komórki
```

7. Wygeneruj i dodaj nowe zdania
8. Wyczyść kolejki zdań
9. Eksportuj wyniki do pliku
0. Wyjdź z systemu

📊 PRZYKŁADOWE UŻYCIE:
- Wybierz 1 (utwórz komórkę)
- Wybierz 7 (wygeneruj zdania)
- Wybierz 3 (uruchom przetwarzanie)
- Wybierz 5 (pokaż status)
- Wybierz 9 (eksportuj wyniki)

📁 PLIKI W PROJEKCIE:
- neuron_system.py - główny system
- neuron_system_v7.py - zaawansowana wersja
- run_neuron_system.py - skrypt uruchomieniowy
- neuron_wyniki_*.json - wyniki przetwarzania

🔧 WYMAGANIA:
- Python 3.8+
- NumPy (zainstaluj: pip install numpy)

📅 Data utworzenia: 2025-12-13 10:26:51

==========
KONIEC PLIKU: README.txt
==========

==========
PLIK 3/14: exporyy.py
==========
Ścieżka: /storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651/exporyy.py
Rozmiar: 10.7 KB
Rozszerzenie: .py
Linie: 342
Znaki: 10774
Encoding: utf-8

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
CODE EXPORTER v1.0
Agreguje wszystkie pliki kodowe z katalogu do jednego dokumentu
tekstowego
```

```
Przygotowane dla narzędzi AI (Claude, ChatGPT, etc.)

Użycie:
    python code_exporter.py

Wynik:
    code_export.txt - wszystkie kody w jednym pliku
"""

import os
import sys
from datetime import datetime
from pathlib import Path
from typing import List, Dict, Tuple

#
==========================================================================
===
# KONFIGURACJA
#
==========================================================================
===

# Rozszerzenia plików do eksportu
CODE_EXTENSIONS = {
    # Python
    '.py', '.pyw', '.pyx', '.pyi',
    # Web
    '.html', '.htm', '.css', '.js', '.jsx', '.ts', '.tsx',
    # Data/Config
    '.json', '.yaml', '.yml', '.xml', '.toml', '.ini', '.cfg',
    # Markdown/Docs
    '.md', '.txt', '.rst', '.adoc',
    # Shell/Batch
    '.sh', '.bash', '.bat', '.cmd', '.ps1',
    # C/C++
    '.c', '.cpp', '.h', '.hpp',
    # Other
    '.sql', '.r', '.java', '.go', '.rs', '.swift'
}

# Foldery do ignorowania
IGNORE_DIRS = {
    '__pycache__',
    '.git',
    '.svn',
    '.hg',
    'node_modules',
    'venv',
    'env',
    '.venv',
    '.env',
    'dist',
    'build',
```

```
    '.idea',
    '.vscode',
    '.pytest_cache',
    'htmlcov',
    '.tox'
}

# Pliki do ignorowania (wzorce)
IGNORE_FILES = {
    '.gitignore',
    '.dockerignore',
    '.DS_Store',
    'Thumbs.db',
    '.pyc',
    'code_export.txt',  # Nie eksportuj samego outputu
    'code_exporter.py'  # Opcjonalnie: nie eksportuj samego skryptu
}

# Maksymalny rozmiar pliku (w bajtach) - 1MB
MAX_FILE_SIZE = 1 * 1024 * 1024

# Nazwa pliku wyjściowego
OUTPUT_FILE = "code_export.txt"

#
==========================================================================
===
# FUNKCJE GŁÓWNE
#
==========================================================================
===

def should_ignore_file(filepath: Path) -> bool:
    """Sprawdza czy plik powinien być ignorowany"""
    # Sprawdź nazwę pliku
    if filepath.name in IGNORE_FILES:
        return True

    # Sprawdź wzorce
    for pattern in IGNORE_FILES:
        if pattern.startswith('.') and filepath.name.endswith(pattern):
            return True

    return False


def should_ignore_dir(dirname: str) -> bool:
    """Sprawdza czy folder powinien być ignorowany"""
    return dirname in IGNORE_DIRS or dirname.startswith('.')


def get_file_size_str(size_bytes: int) -> str:
    """Konwertuje rozmiar na czytelny format"""
    for unit in ['B', 'KB', 'MB', 'GB']:
```

```
        if size_bytes < 1024.0:
            return f"{size_bytes:.1f} {unit}"
        size_bytes /= 1024.0
    return f"{size_bytes:.1f} TB"


def detect_encoding(filepath: Path) -> str:
    """Wykrywa encoding pliku"""
    encodings = ['utf-8', 'utf-8-sig', 'latin-1', 'cp1252', 'iso-8859-1']

    for encoding in encodings:
        try:
            with open(filepath, 'r', encoding=encoding) as f:
                f.read()
            return encoding
        except (UnicodeDecodeError, UnicodeError):
            continue

    return 'utf-8'  # Fallback


def scan_directory(root_path: Path) -> List[Path]:
    """
    Skanuje katalog rekurencyjnie i zwraca listę plików do eksportu
    """
    files_to_export = []

    print(f"🔍 Skanowanie katalogu: {root_path}")
    print(f"📁 Ignorowane foldery: {', '.join(sorted(IGNORE_DIRS))}\n")

    for root, dirs, files in os.walk(root_path):
        # Usuń ignorowane foldery z listy (modyfikacja in-place)
        dirs[:] = [d for d in dirs if not should_ignore_dir(d)]

        for filename in files:
            filepath = Path(root) / filename

            # Sprawdź rozszerzenie
            if filepath.suffix.lower() not in CODE_EXTENSIONS:
                continue

            # Sprawdź ignorowane pliki
            if should_ignore_file(filepath):
                continue

            # Sprawdź rozmiar
            try:
                file_size = filepath.stat().st_size
                if file_size > MAX_FILE_SIZE:
                    print(f"⚠  Pominięto (za duży):
{filepath.relative_to(root_path)} ({get_file_size_str(file_size)})")
                    continue
            except OSError:
```

```
                continue

            files_to_export.append(filepath)

    return sorted(files_to_export)


def read_file_content(filepath: Path) -> Tuple[str, str]:
    """
    Czyta zawartość pliku
    Zwraca: (content, encoding)
    """
    encoding = detect_encoding(filepath)

    try:
        with open(filepath, 'r', encoding=encoding, errors='replace') as
f:
            content = f.read()
        return content, encoding
    except Exception as e:
        return f"[BŁĄD ODCZYTU: {e}]", encoding


def create_export_document(files: List[Path], root_path: Path,
output_file: Path):
    """
    Tworzy dokument eksportowy
    """
    print(f"\n📝 Tworzenie dokumentu eksportowego: {output_file}")
    print(f"📊 Liczba plików: {len(files)}\n")

    total_lines = 0
    total_chars = 0
    stats_by_extension = {}

    with open(output_file, 'w', encoding='utf-8') as out:
        # Header
        out.write("=" * 80 + "\n")
        out.write("CODE EXPORT - AGREGACJA KODÓW\n")
        out.write("=" * 80 + "\n\n")
        out.write(f"📅 Data eksportu: {datetime.now().strftime('%Y-%m-%d
%H:%M:%S')}\n")
        out.write(f"📁 Katalog źródłowy: {root_path.absolute()}\n")
        out.write(f"📊 Liczba plików: {len(files)}\n")
        out.write(f"🔧 Rozszerzenia: {',
'.join(sorted(CODE_EXTENSIONS))}\n")
        out.write("\n" + "=" * 80 + "\n\n")

        # Spis treści
        out.write("📋 SPIS TREŚCI\n")
        out.write("-" * 80 + "\n\n")
        for i, filepath in enumerate(files, 1):
```

```
            rel_path = filepath.relative_to(root_path)
            out.write(f"{i:3d}. {rel_path}\n")
        out.write("\n" + "=" * 80 + "\n\n")

        # Eksport plików
        for i, filepath in enumerate(files, 1):
            rel_path = filepath.relative_to(root_path)
            file_size = filepath.stat().st_size

            print(f"   {i:3d}/{len(files)} | {rel_path}
({get_file_size_str(file_size)})")

            # Header pliku
            out.write("\n" + "=" * 80 + "\n")
            out.write(f"PLIK {i}/{len(files)}: {rel_path}\n")
            out.write("=" * 80 + "\n")
            out.write(f"Ścieżka:    {filepath.absolute()}\n")
            out.write(f"Rozmiar:    {get_file_size_str(file_size)}\n")
            out.write(f"Rozszerzenie: {filepath.suffix}\n")

            # Odczyt zawartości
            content, encoding = read_file_content(filepath)
            line_count = content.count('\n') + 1
            char_count = len(content)

            out.write(f"Linie:      {line_count}\n")
            out.write(f"Znaki:      {char_count}\n")
            out.write(f"Encoding:   {encoding}\n")
            out.write("-" * 80 + "\n\n")

            # Zawartość
            out.write(content)

            # Separator
            out.write("\n\n" + "=" * 80 + "\n")
            out.write(f"KONIEC PLIKU: {rel_path}\n")
            out.write("=" * 80 + "\n\n")

            # Statystyki
            total_lines += line_count
            total_chars += char_count
            ext = filepath.suffix.lower()
            if ext not in stats_by_extension:
                stats_by_extension[ext] = {'count': 0, 'lines': 0,
'chars': 0}
            stats_by_extension[ext]['count'] += 1
            stats_by_extension[ext]['lines'] += line_count
            stats_by_extension[ext]['chars'] += char_count

        # Footer - statystyki
        out.write("\n\n" + "=" * 80 + "\n")
        out.write("📊 STATYSTYKI EKSPORTU\n")
        out.write("=" * 80 + "\n\n")
        out.write(f"Łącznie plików:     {len(files)}\n")
```

```
        out.write(f"Łącznie linii:      {total_lines:,}\n")
        out.write(f"Łącznie znaków:     {total_chars:,}\n")
        out.write(f"Data eksportu:      {datetime.now().strftime('%Y-%m-
%d %H:%M:%S')}\n")
        out.write("\n")

        # Statystyki per rozszerzenie
        out.write("📈 Statystyki według rozszerzeń:\n")
        out.write("-" * 80 + "\n\n")
        for ext in sorted(stats_by_extension.keys()):
            stats = stats_by_extension[ext]
            out.write(f"{ext:10s} | Plików: {stats['count']:3d} | ")
            out.write(f"Linii: {stats['lines']:6,d} | Znaków:
{stats['chars']:8,d}\n")

        out.write("\n" + "=" * 80 + "\n")
        out.write("KONIEC DOKUMENTU EKSPORTOWEGO\n")
        out.write("=" * 80 + "\n")

    return total_lines, total_chars, stats_by_extension


def main():
    """Główna funkcja"""
    print("=" * 80)
    print("🚀 CODE EXPORTER v1.0")
    print("=" * 80)
    print()

    # Katalog roboczy
    root_path = Path.cwd()
    output_path = root_path / OUTPUT_FILE

    # Skanuj katalog
    files = scan_directory(root_path)

    if not files:
        print("❌ Nie znaleziono plików do eksportu!")
        print(f"   Sprawdź czy w katalogu są pliki z rozszerzeniami: {',
'.join(sorted(CODE_EXTENSIONS))}")
        return 1

    print(f"✅ Znaleziono {len(files)} plików do eksportu\n")

    # Potwierdzenie
    print("📋 Lista plików do eksportu:")
    for i, filepath in enumerate(files[:10], 1):
        print(f"   {i}. {filepath.relative_to(root_path)}")
    if len(files) > 10:
        print(f"   ... i {len(files) - 10} więcej")

    print()
    response = input("Kontynuować eksport? (t/n): ").strip().lower()
```

```
    if response not in ['t', 'tak', 'y', 'yes']:
        print("❌ Anulowano")
        return 1

    # Eksport
    try:
        total_lines, total_chars, stats = create_export_document(files,
root_path, output_path)

        # Podsumowanie
        print("\n" + "=" * 80)
        print("✅ EKSPORT ZAKOŃCZONY POMYŚLNIE!")
        print("=" * 80)
        print(f"\n📄 Plik wyjściowy: {output_path}")
        print(f"📊 Rozmiar:
{get_file_size_str(output_path.stat().st_size)}")
        print(f"📝 Plików:         {len(files)}")
        print(f"📏 Linii:          {total_lines:,}")
        print(f"🔤 Znaków:         {total_chars:,}")
        print(f"\n💡 Plik gotowy do użycia z narzędziami AI (Claude,
ChatGPT, etc.)")
        print("=" * 80)

        return 0

    except Exception as e:
        print(f"\n❌ BŁĄD podczas eksportu: {e}")
        import traceback
        traceback.print_exc()
        return 1


if __name__ == "__main__":
    try:
        sys.exit(main())
    except KeyboardInterrupt:
        print("\n\n⚠  Przerwano przez użytkownika")
        sys.exit(1)
```

```
================================================================================
KONIEC PLIKU: exporyy.py
================================================================================
```

```
================================================================================
PLIK 4/14: marge.py
================================================================================
```

Ścieżka:
/storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651/marge.py
Rozmiar: 8.5 KB
Rozszerzenie: .py
Linie: 277
Znaki: 8193
Encoding: utf-8


```
# MIGRATE CORRECT - Skrypt który tworzy projekt we właściwym miejscu
import os
import shutil
import datetime
from pathlib import Path

def find_neuron_project():
    """Znajduje folder z projektem Neuron Swarm"""

    print("🔍 Szukam projektu Neuron Swarm...")

    # Sprawdź typowe lokalizacje
    search_locations = [
        Path.home() / "Desktop",
        Path.home() / "Documents",
        Path.home() / "Downloads",
        Path.home() / "Projects",
        Path.cwd()  # Bieżący folder
    ]

    for location in search_locations:
        if not location.exists():
            continue

        # Szukaj plików projektu
        for item in location.rglob("*"):
            if item.is_file():
                if "neuron_system" in item.name.lower() or
"neuron_wyniki" in item.name:
                    print(f"✅ Znaleziono: {item}")
                    return item.parent

    # Jeśli nie znaleziono, zapytaj użytkownika
    print("❌ Nie znaleziono projektu automatycznie.")
    manual_path = input("Podaj ścieżkę do folderu z projektem (lub
naciśnij Enter dla pulpitu): ").strip()

    if manual_path:
        return Path(manual_path)
    else:
        return Path.home() / "Desktop"

def create_project_copy(source_dir):
```

```
    """Tworzy kopię projektu"""

    # Nazwa nowego projektu
    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    project_name = f"Neuron_Swarm_v7_{timestamp}"

    # Utwórz na Pulpicie
    desktop = Path.home() / "Desktop"
    target_dir = desktop / project_name

    # Sprawdź czy już istnieje
    if target_dir.exists():
        print(f"⚠  Folder {project_name} już istnieje na Pulpicie!")
        choice = input("Nadpisać? (t/n): ").lower().strip()
        if choice != 't':
            counter = 1
            while target_dir.exists():
                target_dir = desktop / f"{project_name}_{counter}"
                counter += 1

    # Utwórz folder
    target_dir.mkdir(parents=True, exist_ok=True)
    print(f"📁 Utworzono: {target_dir}")

    # Skopiuj pliki
    print(f"\n📋 Kopiowanie plików z: {source_dir.name}")

    files_copied = 0
    skipped_folders = ['__pycache__', '.git', 'venv', '.idea']

    for item in source_dir.iterdir():
        if item.name in skipped_folders:
            continue

        try:
            target_path = target_dir / item.name

            if item.is_file():
                # Kopiuj tylko ważne pliki
                if item.suffix in ['.py', '.txt', '.json', '.md',
'.yml']:
                    shutil.copy2(item, target_path)
                    files_copied += 1
                    print(f"   ✓ {item.name}")
            elif item.is_dir():
                # Kopiuj całe foldery
                shutil.copytree(item, target_path,

ignore=shutil.ignore_patterns('__pycache__', '*.pyc'))
                files_copied += 1
                print(f"   📁 {item.name}/")

        except Exception as e:
```

```
            print(f"   ✗ {item.name}: {e}")

    return target_dir, files_copied

def create_essential_files(target_dir):
    """Tworzy niezbędne pliki"""

    print(f"\n📄 Tworzenie plików konfiguracyjnych...")

    # 1. requirements.txt
    requirements = """# Neuron Swarm System v7.0
# Minimalne wymagania
numpy>=1.21.0

# Opcjonalne (do wizualizacji)
# matplotlib>=3.5.0

# Opcjonalne (do zaawansowanej analizy)
# scikit-learn>=1.0.0
# pandas>=1.3.0
"""

    req_path = target_dir / "requirements.txt"
    with open(req_path, 'w', encoding='utf-8') as f:
        f.write(requirements)
    print("   ✅ requirements.txt")

    # 2. run.bat
    run_bat = """@echo off
echo ========================================
echo 🧠 NEURON SWARM SYSTEM v7.0
echo ========================================
echo.
echo Uruchamianie systemu neuronowego...
echo.

REM Sprawdź czy python jest dostępny
where python >nul 2>nul
if %errorlevel% neq 0 (
    echo ✗ Python nie znaleziony!
    echo Zainstaluj Python 3.8+ z python.org
    pause
    exit /b 1
)

REM Uruchom system
python neuron_system.py

if %errorlevel% neq 0 (
    echo.
    echo ⚠  System zakończył się z błędem
    echo Sprawdź czy wszystkie pliki są w folderze
)
```

```
pause
"""

    run_path = target_dir / "run.bat"
    with open(run_path, 'w', encoding='utf-8') as f:
        f.write(run_bat)
    print("   ✅ run.bat (skrypt uruchomieniowy Windows)")

    # 3. README.txt
    readme = f"""NEURON SWARM SYSTEM v7.0
{'='*40}

📋 OPIS SYSTEMU:
System neuronowy przetwarzający zdania w wektory semantyczne.
3 komórki × 4 neurony = 12 neuronów przetwarzających równolegle.

🚀 JAK URUCHOMIĆ:
1. Kliknij dwukrotnie: run.bat
2. Lub w konsoli: python neuron_system.py

📋 MENU SYSTEMU:
1. Utwórz komórkę neuronową
2. Dodaj zdania do przetwarzania
3. Uruchom przetwarzanie w tle
4. Zatrzymaj przetwarzanie
5. Pokaż status wszystkich komórek
6. Pokaż szczegółowy raport komórki
7. Wygeneruj i dodaj nowe zdania
8. Wyczyść kolejki zdań
9. Eksportuj wyniki do pliku
0. Wyjdź z systemu

📊 PRZYKŁADOWE UŻYCIE:
- Wybierz 1 (utwórz komórkę)
- Wybierz 7 (wygeneruj zdania)
- Wybierz 3 (uruchom przetwarzanie)
- Wybierz 5 (pokaż status)
- Wybierz 9 (eksportuj wyniki)

📁 PLIKI W PROJEKCIE:
- neuron_system.py - główny system
- neuron_system_v7.py - zaawansowana wersja
- run_neuron_system.py - skrypt uruchomieniowy
- neuron_wyniki_*.json - wyniki przetwarzania

🔧 WYMAGANIA:
- Python 3.8+
- NumPy (zainstaluj: pip install numpy)

📅 Data utworzenia: {datetime.datetime.now().strftime('%Y-%m-%d
%H:%M:%S')}
```

```
"""

    readme_path = target_dir / "README.txt"
    with open(readme_path, 'w', encoding='utf-8') as f:
        f.write(readme)
    print("   ✅ README.txt")

    return ["requirements.txt", "run.bat", "README.txt"]

def main():
    """Główna funkcja"""

    print("="*60)
    print("□ MIGRACJA PROJEKTU NEURON SWARM - POPRAWIONA")
    print("="*60)

    try:
        # 1. Znajdź źródłowy projekt
        source_dir = find_neuron_project()
        print(f"\n📂 Źródłowy projekt: {source_dir}")

        # Pokaż pliki w źródłowym
        print("📋 Pliki w źródłowym folderze:")
        py_files = [f for f in source_dir.glob("*.py") if f.is_file()]
        for i, file in enumerate(py_files[:10], 1):
            print(f"   {i:2d}. {file.name}")
        if len(py_files) > 10:
            print(f"   ... i {len(py_files) - 10} więcej")

        # 2. Utwórz kopię na Pulpicie
        print(f"\n{'='*40}")
        print("🔄 Tworzenie kopii projektu...")

        target_dir, files_copied = create_project_copy(source_dir)

        if files_copied == 0:
            print("❌ Nie skopiowano żadnych plików!")
            return

        # 3. Utwórz pliki konfiguracyjne
        create_essential_files(target_dir)

        # 4. Podsumowanie
        print(f"\n{'='*60}")
        print("🎉 MIGRACJA ZAKOŃCZONA POMYŚLNIE!")
        print(f"{'='*60}")
        print(f"\n📁 NOWY PROJEKT: {target_dir.name}")
        print(f"📍 LOKALIZACJA: {target_dir}")
        print(f"📄 PLIKÓW: {files_copied}")

        # 5. Instrukcja
        print(f"\n🚀 JAK URUCHOMIĆ:")
```

```
        print(f"   1. Otwórz folder: {target_dir.name}")
        print(f"   2. Kliknij dwukrotnie: run.bat")
        print(f"   3. Lub w konsoli: python neuron_system.py")

        # 6. Otwórz folder
        print(f"\n📋 Przejdź do folderu? (Spowoduje otwarcie
Eksploratora)")
        open_folder = input("   Tak/Nie (t/n): ").lower().strip()

        if open_folder == 't':
            try:
                os.startfile(target_dir)
                print("   ✅ Otwieram folder...")
            except:
                print(f"   ℹ  Folder: {target_dir}")

        print(f"\n{'='*60}")
        print("✅ Gotowe! Projekt jest na Twoim Pulpicie.")
        print(f"{'='*60}")

    except KeyboardInterrupt:
        print("\n\n⚠  Przerwano przez użytkownika")
    except Exception as e:
        print(f"\n❌ Błąd: {e}")
        import traceback
        traceback.print_exc()
        input("\nNaciśnij Enter aby zakończyć...")

if __name__ == "__main__":
    main()
```

```
============================================================================
=======
KONIEC PLIKU: marge.py
============================================================================
=======


============================================================================
=======
PLIK 5/14: neuron_cell.py
============================================================================
=======
Ścieżka:
/storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651/neuron_cell.
py
Rozmiar:    2.5 KB
Rozszerzenie: .py
Linie:      82
Znaki:      2453
Encoding:   utf-8
----------------------------------------------------------------------------
-------
```

```
# ORCHESTRATOR - Monitorowanie i zarządzanie całym rojem

import subprocess
import time
import psutil
import json
from tabulate import tabulate

def launch_cell(cell_number, total_cells=10):
    """Uruchom nową komórkę neuronów"""
    # W rzeczywistości uruchamiamy nowy proces Python
    cmd = ["python", "neuron_cell.py"]

    # Symulacja - tutaj rzeczywiste uruchomienie
    print(f"[ORCHESTRATOR] Uruchamiam komórkę #{cell_number}")

    # Zwróć symulowane dane
    return {
        "id": f"CELL-{cell_number:03d}",
        "pid": 1000 + cell_number,
        "start_time": time.time(),
        "neurons": 5,
        "port": 5555 + cell_number
    }

def monitor_swarm(cells):
    """Monitorowanie wszystkich komórek"""
    data = []

    for cell in cells:
        # Symulacja danych (w rzeczywistości pobierane przez socket)
        cpu = psutil.cpu_percent(interval=0.1)
        mem = psutil.virtual_memory().percent

        cell_data = [
            cell["id"],
            cell["pid"],
            "AKTYWNA" if random.random() > 0.1 else "PAUZA",
            cell["neurons"],
            f"{cpu:.1f}%",
            f"{mem:.1f}%",
            time.strftime("%H:%M:%S", time.localtime(cell["start_time"]))
        ]
        data.append(cell_data)

    print("\n" + "="*80)
    print("🎯 ORCHESTRATOR RÓJU NEURONÓW - LIVE MONITOR")
    print("="*80)

    headers = ["ID Komórki", "PID", "Status", "Neurony", "CPU", "RAM",
"Uruchomiona"]
    print(tabulate(data, headers=headers, tablefmt="grid"))
```

```
    print(f"\n📊 Podsumowanie: {len(cells)} komórek, {len(cells)*5}
neuronów")
    print("   Użyj Ctrl+C aby zatrzymać monitorowanie")

if __name__ == "__main__":
    import random

    print("=== SYSTEM RÓJU NEURONÓW - ORCHESTRATOR ===")
    print("1. Uruchom nową komórkę neuronów")
    print("2. Uruchom wiele komórek (klaster)")
    print("3. Monitoruj istniejące komórki")

    choice = input("Wybierz (1-3): ").strip()

    cells = []

    if choice == "2":
        num_cells = int(input("Ile komórek uruchomić? (1-20): "))
        for i in range(num_cells):
            cells.append(launch_cell(i+1, num_cells))
            time.sleep(0.5)  # Opóźnienie między uruchomieniami

    # Monitorowanie
    try:
        while True:
            monitor_swarm(cells)
            time.sleep(5)  # Odśwież co 5 sekund
    except KeyboardInterrupt:
        print("\n\n⏹ Zatrzymywanie orchestracji...")
```

==========
KONIEC PLIKU: neuron_cell.py
==========

==========
PLIK 6/14: neuron_system_v7.0.py
==========
Ścieżka: /storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651/neuron_system_v7.0.py
Rozmiar: 34.2 KB
Rozszerzenie: .py
Linie: 858
Znaki: 33666
Encoding: utf-8

```
# NEURON SWARM SYSTEM v7.0 - ULEPSZONY
# Dodaje analizę semantyczną, różnorodność zdań i wizualizację

import numpy as np
import time
import random
import threading
from collections import deque, defaultdict
import json
from datetime import datetime
import os
import sys
import re
from typing import List, Dict, Tuple
import hashlib

print("=" * 70)
print("□ SYSTEM NEURONOWY v7.0 - ULEPSZONY")
print("=" * 70)

# === ULEPSZONY GENERATOR ZDAŃ ===
class UlepszonyGenerator:
    """Generuje bardziej różnorodne zdania"""

    def __init__(self):
        self.slownictwo = {
            'system': ['system', 'mechanizm', 'algorytm', 'model',
'architektura'],
            'neuron': ['neuron', 'komórka', 'jednostka', 'moduł',
'element'],
            'przetwarzanie': ['przetwarza', 'analizuje', 'przekształca',
'konwertuje', 'interpretuje'],
            'dane': ['zdania', 'wektory', 'dane', 'teksty',
'informacje'],
            'sposób': ['skutecznie', 'szybko', 'dokładnie',
'inteligentnie', 'automatycznie'],
            'cel': ['analizy', 'klasyfikacji', 'rozumienia',
'przetwarzania', 'optymalizacji']
        }

        self.szablony = [
            "{} {} {} w cel {} {}.",
            "Dla {} {} {} {} i {}.",
            "{} przy użyciu {} {} {} {}.",
            "W procesie {} {} {} poprzez {}.",
            "{} realizuje {} za pomocą {} w {}.",
            "Metoda {} {} {} dla {} {}.",
            "W {} {} jest {} przez {}.",
            "{} umożliwia {} {} w {}.",
            "Algorytm {} {} {} do {}.",
            "System {} {} {} w {}."
        ]
```

```
    def generuj_zdanie(self) -> str:
        """Generuje unikalne zdanie 8-12 słów"""
        szablon = random.choice(self.szablony)

        # Wypełnij szablon
        slowa = []
        for _ in range(szablon.count('{}')):
            kategoria = random.choice(list(self.slownictwo.keys()))
            slowa.append(random.choice(self.slownictwo[kategoria]))

        zdanie = szablon.format(*slowa)

        # Upewnij się o długości 8-12 słów
        wyrazy = zdanie.split()
        if len(wyrazy) < 8:
            dodatek = random.sample(self.slownictwo['dane'], 8 -
len(wyrazy))
            zdanie = zdanie.replace('.', ' ' + ' '.join(dodatek) + '.')
        elif len(wyrazy) > 12:
            zdanie = ' '.join(wyrazy[:12]) + '.'

        return zdanie.capitalize()

    def generuj_partie(self, liczba: int = 10) -> List[str]:
        """Generuje partię unikalnych zdań"""
        zdania = []
        for _ in range(liczba):
            zdania.append(self.generuj_zdanie())
        return zdania

# === NEURON Z ANALIZĄ SEMANTYCZNĄ ===
class NeuronSemantyczny:
    """Neuron z rzeczywistą analizą semantyczną"""

    def __init__(self, neuron_id, neuron_type):
        self.id = neuron_id
        self.type = neuron_type

        # Zaawansowany stan
        self.state = {
            "aktywnosc": random.uniform(0.3, 0.7),
            "energia": 100.0,
            "przetworzone": 0,
            "status": "gotowy",
            "doswiadczenie": 0,
            "specjalizacja": self._init_specjalizacja()
        }

        # Zaawansowana pamięć
        self.pamiec = deque(maxlen=50)
        self.slownik = defaultdict(int)
        self.wzorce = []
        self.wektory_baza = []  # Baza wektorów dla analizy podobieństw
```

```
        # Statystyki
        self.statystyki = {
            "sredni_czas": 0,
            "bledy": 0,
            "unikalne_slowa": 0,
            "wzorce_znalezione": 0
        }

    def _init_specjalizacja(self) -> Dict:
        """Inicjalizuje specjalizację neuronu"""
        specjalizacje = {
            "analiza": {"precyzja": 0.8, "szybkosc": 0.6, "glebia": 0.9},
            "pamiec": {"pojemnosc": 0.9, "trwalosc": 0.8, "dostep": 0.7},
            "logika": {"wnioskowanie": 0.9, "konsekwencja": 0.8,
"abstrakcja": 0.7},
            "kreatywnosc": {"oryginalnosc": 0.9, "elastycznosc": 0.8,
"asocjacja": 0.7}
        }
        return specjalizacje.get(self.type, {"ogolna": 0.5})

    def _analizuj_zdanie(self, zdanie: str) -> Dict:
        """Przeprowadza rzeczywistą analizę zdania"""
        slowa = re.findall(r'\b[a-zA-ZąćęłńóśźżĄĆĘŁŃÓŚŹŻ]+\b',
zdanie.lower())

        # Statystyki podstawowe
        statystyki = {
            "dlugosc": len(zdanie),
            "liczba_slow": len(slowa),
            "srednia_dlugosc_slowa": sum(len(s) for s in slowa) /
max(len(slowa), 1),
            "unikalne_slowa": len(set(slowa)),
            "znaki_specjalne": len(re.findall(r'[.,;:!?]', zdanie))
        }

        # Analiza w zależności od typu neuronu
        if self.type == "analiza":
            analiza = self._analiza_strukturalna(zdanie, slowa)
        elif self.type == "pamiec":
            analiza = self._analiza_leksykalna(zdanie, slowa)
        elif self.type == "logika":
            analiza = self._analiza_logiczna(zdanie, slowa)
        else:  # kreatywnosc
            analiza = self._analiza_kreatywna(zdanie, slowa)

        return {**statystyki, **analiza}

    def _analiza_strukturalna(self, zdanie: str, slowa: List[str]) ->
Dict:
        """Analiza struktury zdania"""
        return {
            "typ_zdania": "pytanie" if "?" in zdanie else "oznajmujące",
            "liczba_czesci": len(re.split(r'[,;:]', zdanie)),
            "poczatek": slowa[0] if slowa else "",
```

```
            "koniec": slowa[-1] if slowa else "",
            "powtorzenia": len(slowa) - len(set(slowa))
        }

    def _analiza_leksykalna(self, zdanie: str, slowa: List[str]) -> Dict:
        """Analiza słownictwa"""
        dlugie_slowa = [s for s in slowa if len(s) > 6]
        krotkie_slowa = [s for s in slowa if len(s) <= 3]

        return {
            "slowa_dlugie": len(dlugie_slowa),
            "slowa_krotkie": len(krotkie_slowa),
            "rozne_koncowki": len(set(s[-2:] for s in slowa if len(s) >
2)),
            "powtarzajace_sie": [s for s in set(slowa) if slowa.count(s)
> 1]
        }

    def _analiza_logiczna(self, zdanie: str, slowa: List[str]) -> Dict:
        """Analiza logiczna"""
        spojniki = ["i", "lub", "albo", "czy", "że", "gdy", "ponieważ"]
        zaprzeczenia = ["nie", "ani", "nigdy", "żaden"]

        return {
            "spojniki": [s for s in slowa if s in spojniki],
            "zaprzeczenia": [s for s in slowa if s in zaprzeczenia],
            "liczba_czynnikow": len([s for s in slowa if
s.endswith(('nie', 'ość', 'stwo'))]),
            "relacje": len([s for s in slowa if s in ["z", "do", "od",
"przez", "dla"]])
        }

    def _analiza_kreatywna(self, zdanie: str, slowa: List[str]) -> Dict:
        """Analiza kreatywna"""
        unikalne_kombinacje = []
        for i in range(len(slowa) - 1):
            kombinacja = f"{slowa[i]}_{slowa[i+1]}"
            unikalne_kombinacje.append(kombinacja)

        return {
            "unikalne_kombinacje": len(set(unikalne_kombinacje)),
            "metafory": len([s for s in slowa if s in ["jak", "niczym",
"podobny", "przypomina"]]),
            "nowe_skojarzenia": random.sample(slowa, min(3, len(slowa)))
if slowa else [],
            "ocena_oryginalnosci": random.uniform(0.3, 0.9)
        }

    def _generuj_wektor(self, analiza: Dict) -> List[float]:
        """Generuje semantyczny wektor na podstawie analizy"""
        wektor = np.zeros(12)  # 12-wymiarowy wektor

        # Wymiary 0-3: Podstawowe statystyki
        wektor[0] = min(analiza.get("dlugosc", 0) / 200, 1.0)
```

```
        wektor[1] = min(analiza.get("liczba_slow", 0) / 15, 1.0)
        wektor[2] = analiza.get("unikalne_slowa", 0) /
max(analiza.get("liczba_slow", 1), 1)
        wektor[3] = min(analiza.get("znaki_specjalne", 0) / 5, 1.0)

        # Wymiary 4-7: Specjalizacja neuronu
        if self.type == "analiza":
            wektor[4] = 1.0 if analiza.get("typ_zdania") == "pytanie"
else 0.5
            wektor[5] = min(analiza.get("liczba_czesci", 0) / 3, 1.0)
            wektor[6] = analiza.get("powtorzenia", 0) /
max(analiza.get("liczba_slow", 1), 1)
        elif self.type == "pamiec":
            wektor[4] = min(analiza.get("slowa_dlugie", 0) / 5, 1.0)
            wektor[5] = min(analiza.get("slowa_krotkie", 0) / 5, 1.0)
            wektor[6] = min(analiza.get("rozne_koncowki", 0) / 10, 1.0)
        elif self.type == "logika":
            wektor[4] = min(len(analiza.get("spojniki", [])) / 3, 1.0)
            wektor[5] = min(len(analiza.get("zaprzeczenia", [])) / 2,
1.0)
            wektor[6] = min(analiza.get("liczba_czynnikow", 0) / 5, 1.0)
        else:  # kreatywnosc
            wektor[4] = min(analiza.get("unikalne_kombinacje", 0) / 10,
1.0)
            wektor[5] = min(analiza.get("metafory", 0) / 2, 1.0)
            wektor[6] = analiza.get("ocena_oryginalnosci", 0.5)

        # Wymiary 8-11: Unikalność i sygnatura
        wektor[7] = random.uniform(0, 0.1)  # Szum
        wektor[8] = hash(zdanie) % 1000 / 1000 if "zdanie" in locals()
else 0.5
        wektor[9] = self.state["aktywnosc"]
        wektor[10] = self.state["doswiadczenie"] / 100
        wektor[11] = sum(wektor[:11]) / 11  # Średnia jako podsumowanie

        return wektor.tolist()

    def _znajdz_podobienstwa(self, nowy_wektor: List[float]) -> Dict:
        """Znajduje podobieństwa z istniejącymi wektorami"""
        if not self.wektory_baza:
            return {"podobne": 0, "najbardziej_podobny": None,
"srednie_podobienstwo": 0}

        podobienstwa = []
        for i, stary_wektor in enumerate(self.wektory_baza):
            # Prosta miara podobieństwa (odległość euklidesowa odwrotnie
proporcjonalna)
            dystans = np.linalg.norm(np.array(nowy_wektor) -
np.array(stary_wektor))
            podobienstwo = 1.0 / (1.0 + dystans)
            podobienstwa.append((i, podobienstwo))

        if podobienstwa:
            najbardziej_podobny = max(podobienstwa, key=lambda x: x[1])
```

```
            srednie = sum(p[1] for p in podobienstwa) / len(podobienstwa)

            return {
                "podobne": len([p for p in podobienstwa if p[1] > 0.7]),
                "najbardziej_podobny": najbardziej_podobny,
                "srednie_podobienstwo": srednie
            }

        return {"podobne": 0, "najbardziej_podobny": None,
"srednie_podobienstwo": 0}

    def przetworz_zdanie(self, zdanie: str) -> Dict:
        """Przetwarza zdanie z pełną analizą"""
        start_czas = time.time()

        # Sprawdź długość
        slowa = zdanie.split()
        if not 8 <= len(slowa) <= 12:
            return None

        try:
            # 1. Analiza semantyczna
            analiza = self._analizuj_zdanie(zdanie)

            # 2. Generuj wektor
            wektor = self._generuj_wektor(analiza)

            # 3. Znajdź podobieństwa
            podobienstwa = self._znajdz_podobienstwa(wektor)

            # 4. Zaktualizuj bazę wektorów
            self.wektory_baza.append(wektor)
            if len(self.wektory_baza) > 100:
                self.wektory_baza.pop(0)

            # 5. Aktualizuj słownik
            for slowo in re.findall(r'\b[a-zA-Ząćęłńóśźż]+\b',
zdanie.lower()):
                self.slownik[slowo] += 1

            # 6. Oblicz czas
            czas_przetwarzania = time.time() - start_czas

            # 7. Przygotuj wynik
            wynik = {
                "zdanie": zdanie,
                "wektor": wektor,
                "analiza": analiza,
                "podobienstwa": podobienstwa,
                "czas": czas_przetwarzania,
                "slowa": len(slowa),
                "typ": self.type,
                "timestamp": time.time(),
                "neuron_id": self.id,
```

```
                "doswiadczenie_neuronu": self.state["doswiadczenie"]
            }

            # 8. Zaktualizuj pamięć
            self.pamiec.append(wynik)

            # 9. Aktualizuj stan neuronu
            self._aktualizuj_stan(czas_przetwarzania, analiza)

            # 10. Znajdź wzorce (co 10 przetworzonych zdań)
            if self.state["przetworzone"] % 10 == 0:
                self._znajdz_wzorce()

            return wynik

        except Exception as e:
            self.statystyki["bledy"] += 1
            print(f"[{self.id}] Błąd przetwarzania: {e}")
            return None

    def _aktualizuj_stan(self, czas: float, analiza: Dict):
        """Aktualizuje stan neuronu po przetworzeniu"""
        self.state["przetworzone"] += 1
        self.state["doswiadczenie"] = min(100,
self.state["doswiadczenie"] + 0.5)

        # Energia zależna od złożoności
        zlozonosc = analiza.get("unikalne_slowa", 0) /
max(analiza.get("liczba_slow", 1), 1)
        self.state["energia"] = max(0, self.state["energia"] - (0.5 +
zlozonosc))

        # Aktywność rośnie z doświadczeniem
        self.state["aktywnosc"] = min(1.0, self.state["aktywnosc"] +
0.01)

        # Aktualizuj statystyki
        self.statystyki["sredni_czas"] = (
            self.statystyki["sredni_czas"] * (self.state["przetworzone"]
- 1) + czas
        ) / self.state["przetworzone"]

        self.statystyki["unikalne_slowa"] = len(self.slownik)

        # Jeśli energia niska, neuron potrzebuje odpoczynku
        if self.state["energia"] < 20:
            self.state["status"] = "odpoczynek"
            self.state["energia"] = min(100, self.state["energia"] + 30)
        else:
            self.state["status"] = "gotowy"

    def _znajdz_wzorce(self):
        """Automatyczne znajdowanie wzorców w danych"""
        if len(self.wektory_baza) < 10:
```

```
            return

        # Prosta analiza klastrów (dla przykładu)
        ostatnie_wektory = self.wektory_baza[-10:]

        # Znajdź podobne grupy
        grupy = []
        for i, v1 in enumerate(ostatnie_wektory):
            grupa = [i]
            for j, v2 in enumerate(ostatnie_wektory):
                if i != j:
                    dystans = np.linalg.norm(np.array(v1) - np.array(v2))
                    if dystans < 0.5:  # Próg podobieństwa
                        grupa.append(j)

            if len(grupa) >= 3 and grupa not in grupy:
                grupy.append(grupa)

        if grupy:
            self.statystyki["wzorce_znalezione"] += len(grupy)
            self.wzorce.append({
                "timestamp": time.time(),
                "liczba_wzorców": len(grupy),
                "rozmiar_grup": [len(g) for g in grupy]
            })

    def status(self) -> Dict:
        """Zwraca szczegółowy status neuronu"""
        return {
            "id": self.id,
            "typ": self.type,
            "stan": self.state,
            "statystyki": self.statystyki,
            "pamiec": len(self.pamiec),
            "slownik": len(self.slownik),
            "wektory_w_bazie": len(self.wektory_baza),
            "wzorce": len(self.wzorce),
            "specjalizacja": self.state["specjalizacja"]
        }

# === ULEPSZONA KOMÓRKA ===
class KomorkaV7:
    """Ulepszona komórka neuronowa"""

    def __init__(self, komorka_id):
        self.id = komorka_id
        self.neurony = []
        self.kolejka = deque(maxlen=100)
        self.wyniki = []
        self.historia = []
        self.generator = UlepszonyGenerator()

        # Tworzenie 4 zaawansowanych neuronów
        typy = ["analiza", "pamiec", "logika", "kreatywnosc"]
```

```
        for i, typ in enumerate(typy):
            neuron_id = f"{komorka_id}-N{i+1}-V7"
            self.neurony.append(NeuronSemantyczny(neuron_id, typ))

        print(f"[{self.id}] Komórka v7.0 utworzona z {len(self.neurony)}
neuronami")

    def dodaj_zdania(self, zdania):
        """Dodaje zdania do kolejki"""
        dodane = 0
        for zdanie in zdania:
            slowa = zdanie.split()
            if 8 <= len(slowa) <= 12:
                self.kolejka.append(zdanie)
                dodane += 1

        self.historia.append({
            "czas": time.time(),
            "akcja": "dodano_zdania",
            "liczba": dodane,
            "kolejka": len(self.kolejka)
        })

        return dodane

    def przetworz_partie(self, rozmiar=3):
        """Przetwarza partię zdań z inteligentnym wyborem neuronu"""
        wyniki_partii = []

        for _ in range(min(rozmiar, len(self.kolejka))):
            zdanie = self.kolejka.popleft()

            # Inteligentny wybór neuronu
            if "?" in zdanie or "czy" in zdanie.lower():
                neuron = next((n for n in self.neurony if n.type ==
"logika"), self.neurony[0])
            elif len(zdanie) > 100:
                neuron = next((n for n in self.neurony if n.type ==
"analiza"), self.neurony[0])
            elif len(set(zdanie.lower().split())) > 8:
                neuron = next((n for n in self.neurony if n.type ==
"pamiec"), self.neurony[0])
            else:
                # Wybierz neuron z najwyższą energią
                neuron = max(self.neurony, key=lambda n:
n.state["energia"])

            # Przetwórz
            wynik = neuron.przetworz_zdanie(zdanie)
            if wynik:
                wynik["komorka"] = self.id
                wyniki_partii.append(wynik)
                self.wyniki.append(wynik)
```

```
        if wyniki_partii:
            self.historia.append({
                "czas": time.time(),
                "akcja": "przetworzono",
                "liczba": len(wyniki_partii),
                "kolejka": len(self.kolejka)
            })

        return wyniki_partii

    def status_komorki(self):
        """Szczegółowy status komórki"""
        statusy_neuronow = [n.status() for n in self.neurony]

        # Statystyki komórki
        lacznie_przetworzone = sum(n.state["przetworzone"] for n in
self.neurony)
        srednia_energia = np.mean([n.state["energia"] for n in
self.neurony])
        laczny_slownik = sum(len(n.slownik) for n in self.neurony)

        # Efektywność
        if len(self.historia) > 1:
            czas_pracy = self.historia[-1]["czas"] -
self.historia[0]["czas"]
            efektywnosc = lacznie_przetworzone / max(czas_pracy, 1)
        else:
            efektywnosc = 0

        return {
            "id_komorki": self.id,
            "wersja": "7.0",
            "neurony": len(self.neurony),
            "kolejka": len(self.kolejka),
            "przetworzone": len(self.wyniki),
            "statystyki": {
                "lacznie_przetworzone": lacznie_przetworzone,
                "srednia_energia": srednia_energia,
                "laczny_slownik": laczny_slownik,
                "efektywnosc": efektywnosc,
                "historia_akcji": len(self.historia)
            },
            "neurony_status": statusy_neuronow
        }

    def raport_szczegolowy(self):
        """Generuje szczegółowy raport"""
        status = self.status_komorki()

        raport = f"""
        📊 SZCZEGÓŁOWY RAPORT KOMÓRKI {self.id} v7.0
        {'='*50}

        📈 STATYSTYKI OGÓLNE:
```

```
        • Neurony: {status['neurony']}
        • W kolejce: {status['kolejka']} zdań
        • Przetworzone: {status['przetworzone']} zdań
        • Łącznie neuronów:
{status['statystyki']['lacznie_przetworzone']}
        • Średnia energia: {status['statystyki']['srednia_energia']:.1f}%
        • Słownik: {status['statystyki']['laczny_slownik']} unikalnych
słów
        • Efektywność: {status['statystyki']['efektywnosc']:.2f}
zdania/sekundę

        □ NEURONY:
        """

        for neuron_status in status["neurony_status"]:
            spec = neuron_status.get("specjalizacja", {})
            spec_str = ", ".join(f"{k}:{v:.2f}" for k, v in spec.items())

            raport += f"""
            {neuron_status['id']} [{neuron_status['typ']:12}]
              Stan: {neuron_status['stan']['status']}
              Energia: {neuron_status['stan']['energia']:6.1f}%
              Doświadczenie: {neuron_status['stan']['doswiadczenie']:.1f}
              Przetworzone: {neuron_status['stan']['przetworzone']:4d}
              Pamięć: {neuron_status['pamiec']:3d} wpisów
              Słownik: {neuron_status['slownik']:4d} słów
              Wektory: {neuron_status['wektory_w_bazie']:3d}
              Wzorce: {neuron_status['wzorce']:2d}
              Specjalizacja: {spec_str}
            """

        # Ostatnie wyniki
        if self.wyniki:
            raport += f"\n\n📝 OSTATNIE WYNIKI (3):"
            for wynik in self.wyniki[-3:]:
                podobne = wynik.get("podobienstwa", {}).get("podobne", 0)
                raport += f"""
                • {wynik['zdanie'][:60]}...
                  Neuron: {wynik['typ']} (podobne: {podobne})
                  Czas: {wynik['czas']:.3f}s, Słowa: {wynik['slowa']}
                """

        return raport

# === SYSTEM Z WIZUALIZACJĄ ===
class SystemV7:
    """System v7.0 z wizualizacją i zaawansowanymi funkcjami"""

    def __init__(self):
        self.komorki = []
        self.generator = UlepszonyGenerator()
        self.uruchomiony = False
        self.watek_przetwarzania = None
        self.historia_systemu = []
```

```
    print("\n" + "="*60)
    print("□  SYSTEM NEURONOWY v7.0 - ULEPSZONY")
    print("="*60)

def pokaz_menu(self):
    """Główne menu systemu"""
    while True:
        print("\n" + "="*50)
        print("📋 MENU GŁÓWNE v7.0")
        print("="*50)
        print("1.  Utwórz nową komórkę (v7.0)")
        print("2.  Dodaj zdania do przetwarzania")
        print("3.  Wygeneruj i dodaj zdania")
        print("4.  Uruchom przetwarzanie w tle")
        print("5.  Zatrzymaj przetwarzanie")
        print("6.  Pokaż status wszystkich komórek")
        print("7.  Szczegółowy raport komórki")
        print("8.  Analizuj wyniki")
        print("9.  Eksportuj dane (JSON)")
        print("10. Eksportuj raport (TXT)")
        print("11. Wyczyść system")
        print("12. Test wydajności")
        print("0.  Wyjdź")
        print("-"*50)

        try:
            wybor = input("Wybierz opcję (0-12): ").strip()

            if wybor == "0":
                self._wyjscie()
                break
            elif wybor == "1":
                self._opcja_1()
            elif wybor == "2":
                self._opcja_2()
            elif wybor == "3":
                self._opcja_3()
            elif wybor == "4":
                self._opcja_4()
            elif wybor == "5":
                self._opcja_5()
            elif wybor == "6":
                self._opcja_6()
            elif wybor == "7":
                self._opcja_7()
            elif wybor == "8":
                self._opcja_8()
            elif wybor == "9":
                self._opcja_9()
            elif wybor == "10":
                self._opcja_10()
            elif wybor == "11":
                self._opcja_11()
```

```
                elif wybor == "12":
                    self._opcja_12()
                else:
                    print("❌ Nieprawidłowy wybór")

            except KeyboardInterrupt:
                print("\n\n⏸  Przerwano")
                self._wyjscie()
                break
            except Exception as e:
                print(f"❌ Błąd: {e}")

    # OPCJE (podobne ale ulepszone) - pokażę tylko nowe opcje 8, 10, 11,
12

    def _opcja_8(self):
        """8. Analizuj wyniki"""
        if not self.komorki or all(len(k.wyniki) == 0 for k in
self.komorki):
            print("❌ Brak wyników do analizy")
            return

        print("\n" + "="*50)
        print("📊 ANALIZA WYNIKÓW")
        print("="*50)

        # Zbierz wszystkie wyniki
        wszystkie_wyniki = []
        for komorka in self.komorki:
            wszystkie_wyniki.extend(komorka.wyniki)

        if not wszystkie_wyniki:
            print("⚠️  Brak danych do analizy")
            return

        # Podstawowe statystyki
        print(f"\n📈 STATYSTYKI OGÓLNE:")
        print(f"• Łączna liczba wyników: {len(wszystkie_wyniki)}")
        print(f"• Liczba komórek: {len(self.komorki)}")

        # Średni czas przetwarzania
        czasy = [w.get("czas", 0) for w in wszystkie_wyniki]
        sredni_czas = sum(czasy) / len(czasy)
        print(f"• Średni czas przetwarzania: {sredni_czas:.3f}s")
        print(f"• Najkrótszy czas: {min(czasy):.3f}s")
        print(f"• Najdłuższy czas: {max(czasy):.3f}s")

        # Rozkład typów neuronów
        typy_neuronow = [w.get("typ", "nieznany") for w in
wszystkie_wyniki]
        licznik_typow = {}
        for typ in typy_neuronow:
            licznik_typow[typ] = licznik_typow.get(typ, 0) + 1
```

```
        print(f"\n□ ROZKŁAD TYPÓW NEURONÓW:")
        for typ, liczba in licznik_typow.items():
            procent = (liczba / len(wszystkie_wyniki)) * 100
            print(f"• {typ:12}: {liczba:3d} ({procent:.1f}%)")

        # Analiza długości zdań
        dlugosci = [w.get("slowa", 0) for w in wszystkie_wyniki]
        print(f"\n📏 DŁUGOŚĆ ZDAŃ:")
        print(f"• Średnia długość: {np.mean(dlugosci):.1f} słów")
        print(f"• Zakres: {min(dlugosci)}-{max(dlugosci)} słów")
        print(f"• Mediana: {np.median(dlugosci):.1f} słów")

        # Unikalne słowa
        wszystkie_slowa = set()
        for wynik in wszystkie_wyniki:
            zdanie = wynik.get("zdanie", "")
            slowa = re.findall(r'\b[a-zA-Zącęłńóśźż]+\b',
zdanie.lower())
            wszystkie_slowa.update(slowa)

        print(f"\n📚 SŁOWNICTWO:")
        print(f"• Unikalnych słów: {len(wszystkie_slowa)}")
        print(f"• Przykładowe słowa: {',
'.join(list(wszystkie_slowa)[:10])}")

        # Analiza podobieństw
        if len(wszystkie_wyniki) > 1:
            print(f"\n🔗 ANALIZA PODOBIEŃSTW:")
            liczba_podobnych = sum(
                1 for w in wszystkie_wyniki
                if w.get("podobienstwa", {}).get("podobne", 0) > 0
            )
            print(f"• Zdań z podobieństwami: {liczba_podobnych}")
            print(f"• Procent:
{(liczba_podobnych/len(wszystkie_wyniki))*100:.1f}%")

        # Sugestie
        print(f"\n💡 SUGESTIE:")
        if sredni_czas > 0.1:
            print("• Czas przetwarzania jest wysoki - rozważ
optymalizację")
        if max(czasy) > 0.2:
            print("• Niektóre zdania przetwarzają się bardzo długo")
        if len(wszystkie_slowa) < 50:
            print("• Słownik jest mały - dodaj bardziej różnorodne
zdania")

    def _opcja_10(self):
        """10. Eksportuj raport (TXT)"""
        if not self.komorki:
            print("❌ Brak komórek do raportowania")
            return
```

```
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        nazwa_pliku = f"neuron_raport_{timestamp}.txt"

        with open(nazwa_pliku, 'w', encoding='utf-8') as f:
            f.write("="*70 + "\n")
            f.write("RAPORT SYSTEMU NEURONOWEGO v7.0\n")
            f.write("="*70 + "\n\n")
            f.write(f"Data generowania: {datetime.now().strftime('%Y-%m-
%d %H:%M:%S')}\n")
            f.write(f"Liczba komórek: {len(self.komorki)}\n")
            f.write(f"Łączna liczba neuronów: {len(self.komorki) *
4}\n\n")

            laczne_wyniki = sum(len(k.wyniki) for k in self.komorki)
            f.write(f"Łącznie przetworzonych zdań: {laczne_wyniki}\n")

            for komorka in self.komorki:
                f.write("\n" + "="*60 + "\n")
                f.write(f"KOMÓRKA: {komorka.id}\n")
                f.write("="*60 + "\n")

                status = komorka.status_komorki()
                f.write(f"\nStatystyki:\n")
                f.write(f"• Neurony: {status['neurony']}\n")
                f.write(f"• W kolejce: {status['kolejka']}\n")
                f.write(f"• Przetworzone: {status['przetworzone']}\n")
                f.write(f"• Średnia energia:
{status['statystyki']['srednia_energia']:.1f}%\n")
                f.write(f"• Słownik:
{status['statystyki']['laczny_slownik']} słów\n")

                f.write(f"\nNeurony:\n")
                for neuron_status in status['neurony_status']:
                    f.write(f"• {neuron_status['id']}: ")
                    f.write(f"E={neuron_status['stan']['energia']:.1f}%,
")
                    f.write(f"P={neuron_status['stan']['przetworzone']},
")
                    f.write(f"S={neuron_status['slownik']} słów\n")

                if komorka.wyniki:
                    f.write(f"\nOstatnie wyniki:\n")
                    for wynik in komorka.wyniki[-3:]:
                        f.write(f"• {wynik['zdanie'][:50]}...\n")
                        f.write(f"  [{wynik['typ']}]
{wynik['czas']:.3f}s, {wynik['slowa']} słów\n")

            f.write("\n" + "="*70 + "\n")
            f.write("KONIEC RAPORTU\n")
            f.write("="*70 + "\n")

        print(f"✅ Raport zapisany do: {nazwa_pliku}")
```

```
        print(f"   • Rozmiar: {os.path.getsize(nazwa_pliku) / 1024:.1f}
KB")

    def _opcja_11(self):
        """11. Wyczyść system"""
        if not self.komorki:
            print("ℹ  System jest już pusty")
            return

        potwierdzenie = input("Czy na pewno wyczyścić CAŁY system? (t/n):
").strip().lower()

        if potwierdzenie == 't':
            # Zatrzymaj przetwarzanie jeśli działa
            if self.uruchomiony:
                self.uruchomiony = False
                if self.watek_przetwarzania:
                    self.watek_przetwarzania.join(timeout=1)

            # Podsumowanie przed czyszczeniem
            laczne_wyniki = sum(len(k.wyniki) for k in self.komorki)
            laczny_slownik = sum(
                sum(len(n.slownik) for n in k.neurony)
                for k in self.komorki
            )

            # Wyczyść
            self.komorki.clear()
            self.historia_systemu.append({
                "czas": time.time(),
                "akcja": "system_wyczyszczony",
                "wyniki_usuniete": laczne_wyniki,
                "slownik_usuniety": laczny_slownik
            })

            print(f"✅ System wyczyszczony")
            print(f"   • Usunięte komórki: {len(self.komorki)}")
            print(f"   • Utracone wyniki: {laczne_wyniki}")
            print(f"   • Utracony słownik: {laczny_slownik} słów")
        else:
            print("❌ Anulowano czyszczenie")

    def _opcja_12(self):
        """12. Test wydajności"""
        print("\n" + "="*50)
        print("⚡ TEST WYDAJNOŚCI SYSTEMU")
        print("="*50)

        # Utwórz tymczasową komórkę do testu
        test_komorka = KomorkaV7("TEST-WYDAJNOSCI")

        # Wygeneruj testowe zdania
        print("\n🔧 Przygotowanie testu...")
```

```
        test_zdania = self.generator.generuj_partie(20)
        test_komorka.dodaj_zdania(test_zdania)

        # Test 1: Przetwarzanie pojedynczych zdań
        print("\n📊 TEST 1: Przetwarzanie pojedyncze")
        czasy = []
        for zdanie in test_zdania[:5]:
            start = time.time()
            wynik = test_komorka.neurony[0].przetworz_zdanie(zdanie)
            czas = time.time() - start
            czasy.append(czas)
            print(f"   • {zdanie[:40]}...: {czas:.3f}s")

        srednia_poj = sum(czasy) / len(czasy)
        print(f"   ŚREDNIA: {srednia_poj:.3f}s na zdanie")

        # Test 2: Przetwarzanie partii
        print("\n📊 TEST 2: Przetwarzanie partii (5 zdań)")
        start = time.time()
        wyniki = []
        for _ in range(5):
            if test_komorka.kolejka:
                wynik = test_komorka.przetworz_partie(1)
                if wynik:
                    wyniki.extend(wynik)
        czas_partii = time.time() - start

        print(f"   • Czas całkowity: {czas_partii:.3f}s")
        print(f"   • Średnia: {czas_partii/len(wyniki):.3f}s na zdanie")

        # Test 3: Współbieżność (symulowana)
        print("\n📊 TEST 3: Symulacja współbieżności")
        if len(test_komorka.neurony) >= 4:
            czasy_neuronow = []
            for i, neuron in enumerate(test_komorka.neurony[:4]):
                start = time.time()
                neuron.przetworz_zdanie(test_zdania[i])
                czas = time.time() - start
                czasy_neuronow.append(czas)
                print(f"   • Neuron {neuron.type}: {czas:.3f}s")

            print(f"   • Różnica najszybszy-najwolniejszy:
{max(czasy_neuronow)-min(czasy_neuronow):.3f}s")

        # Podsumowanie
        print("\n" + "="*50)
        print("📈 PODSUMOWANIE WYDAJNOŚCI:")
        print(f"• Pojedyncze zdanie: {srednia_poj:.3f}s")
        print(f"• Partia (5 zdań): {czas_partii:.3f}s
({czas_partii/5:.3f}s na zdanie)")
        print(f"• Zalecana wielkość partii: 3-5 zdań")

        # Usuń tymczasową komórkę
```

```
        del test_komorka
        print("\n✅ Test zakończony")

# === URUCHOMIENIE ===
if __name__ == "__main__":
    print("⚡ Ładowanie Systemu Neuronowego v7.0...")
    time.sleep(0.5)

    system = SystemV7()
    system.pokaz_menu()
```

```
==========================================================================
=======
KONIEC PLIKU: neuron_system_v7.0.py
==========================================================================
=======


==========================================================================
=======
PLIK 7/14: neuron_system_v7.py
==========================================================================
=======
Ścieżka:
/storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651/neuron_syste
m_v7.py
Rozmiar:    34.2 KB
Rozszerzenie: .py
Linie:      858
Znaki:      33666
Encoding:   utf-8
---------------------------------------------------------------------------
-------
```

```
# NEURON SWARM SYSTEM v7.0 - ULEPSZONY
# Dodaje analizę semantyczną, różnorodność zdań i wizualizację

import numpy as np
import time
import random
import threading
from collections import deque, defaultdict
import json
from datetime import datetime
import os
import sys
import re
from typing import List, Dict, Tuple
import hashlib

print("=" * 70)
print("□ SYSTEM NEURONOWY v7.0 - ULEPSZONY")
print("=" * 70)
```

```
# === ULEPSZONY GENERATOR ZDAŃ ===
class UlepszonyGenerator:
    """Generuje bardziej różnorodne zdania"""

    def __init__(self):
        self.slownictwo = {
            'system': ['system', 'mechanizm', 'algorytm', 'model',
'architektura'],
            'neuron': ['neuron', 'komórka', 'jednostka', 'moduł',
'element'],
            'przetwarzanie': ['przetwarza', 'analizuje', 'przekształca',
'konwertuje', 'interpretuje'],
            'dane': ['zdania', 'wektory', 'dane', 'teksty',
'informacje'],
            'sposób': ['skutecznie', 'szybko', 'dokładnie',
'inteligentnie', 'automatycznie'],
            'cel': ['analizy', 'klasyfikacji', 'rozumienia',
'przetwarzania', 'optymalizacji']
        }

        self.szablony = [
            "{} {} {} w cel {} {}.",
            "Dla {} {} {} {} i {}.",
            "{} przy użyciu {} {} {} {}.",
            "W procesie {} {} {} poprzez {}.",
            "{} realizuje {} za pomocą {} w {}.",
            "Metoda {} {} {} dla {} {}.",
            "W {} {} jest {} przez {}.",
            "{} umożliwia {} {} w {}.",
            "Algorytm {} {} {} do {}.",
            "System {} {} {} w {}."
        ]

    def generuj_zdanie(self) -> str:
        """Generuje unikalne zdanie 8-12 słów"""
        szablon = random.choice(self.szablony)

        # Wypełnij szablon
        slowa = []
        for _ in range(szablon.count('{}')):
            kategoria = random.choice(list(self.slownictwo.keys()))
            slowa.append(random.choice(self.slownictwo[kategoria]))

        zdanie = szablon.format(*slowa)

        # Upewnij się o długości 8-12 słów
        wyrazy = zdanie.split()
        if len(wyrazy) < 8:
            dodatek = random.sample(self.slownictwo['dane'], 8 -
len(wyrazy))
            zdanie = zdanie.replace('.', ' ' + ' '.join(dodatek) + '.')
        elif len(wyrazy) > 12:
            zdanie = ' '.join(wyrazy[:12]) + '.'
```

```
        return zdanie.capitalize()

    def generuj_partie(self, liczba: int = 10) -> List[str]:
        """Generuje partię unikalnych zdań"""
        zdania = []
        for _ in range(liczba):
            zdania.append(self.generuj_zdanie())
        return zdania

# === NEURON Z ANALIZĄ SEMANTYCZNĄ ===
class NeuronSemantyczny:
    """Neuron z rzeczywistą analizą semantyczną"""

    def __init__(self, neuron_id, neuron_type):
        self.id = neuron_id
        self.type = neuron_type

        # Zaawansowany stan
        self.state = {
            "aktywnosc": random.uniform(0.3, 0.7),
            "energia": 100.0,
            "przetworzone": 0,
            "status": "gotowy",
            "doswiadczenie": 0,
            "specjalizacja": self._init_specjalizacja()
        }

        # Zaawansowana pamięć
        self.pamiec = deque(maxlen=50)
        self.slownik = defaultdict(int)
        self.wzorce = []
        self.wektory_baza = []  # Baza wektorów dla analizy podobieństw

        # Statystyki
        self.statystyki = {
            "sredni_czas": 0,
            "bledy": 0,
            "unikalne_slowa": 0,
            "wzorce_znalezione": 0
        }

    def _init_specjalizacja(self) -> Dict:
        """Inicjalizuje specjalizację neuronu"""
        specjalizacje = {
            "analiza": {"precyzja": 0.8, "szybkosc": 0.6, "glebia": 0.9},
            "pamiec": {"pojemnosc": 0.9, "trwalosc": 0.8, "dostep": 0.7},
            "logika": {"wnioskowanie": 0.9, "konsekwencja": 0.8,
"abstrakcja": 0.7},
            "kreatywnosc": {"oryginalnosc": 0.9, "elastycznosc": 0.8,
"asocjacja": 0.7}
        }
        return specjalizacje.get(self.type, {"ogolna": 0.5})

    def _analizuj_zdanie(self, zdanie: str) -> Dict:
```

```
        """Przeprowadza rzeczywistą analizę zdania"""
        slowa = re.findall(r'\b[a-zA-ZąćęłńóśźżĄĆĘŁŃÓŚŹŻ]+\b',
zdanie.lower())

        # Statystyki podstawowe
        statystyki = {
            "dlugosc": len(zdanie),
            "liczba_slow": len(slowa),
            "srednia_dlugosc_slowa": sum(len(s) for s in slowa) /
max(len(slowa), 1),
            "unikalne_slowa": len(set(slowa)),
            "znaki_specjalne": len(re.findall(r'[.,;:!?]', zdanie))
        }

        # Analiza w zależności od typu neuronu
        if self.type == "analiza":
            analiza = self._analiza_strukturalna(zdanie, slowa)
        elif self.type == "pamiec":
            analiza = self._analiza_leksykalna(zdanie, slowa)
        elif self.type == "logika":
            analiza = self._analiza_logiczna(zdanie, slowa)
        else:  # kreatywnosc
            analiza = self._analiza_kreatywna(zdanie, slowa)

        return {**statystyki, **analiza}

    def _analiza_strukturalna(self, zdanie: str, slowa: List[str]) ->
Dict:
        """Analiza struktury zdania"""
        return {
            "typ_zdania": "pytanie" if "?" in zdanie else "oznajmujące",
            "liczba_czesci": len(re.split(r'[,;:]', zdanie)),
            "poczatek": slowa[0] if slowa else "",
            "koniec": slowa[-1] if slowa else "",
            "powtorzenia": len(slowa) - len(set(slowa))
        }

    def _analiza_leksykalna(self, zdanie: str, slowa: List[str]) -> Dict:
        """Analiza słownictwa"""
        dlugie_slowa = [s for s in slowa if len(s) > 6]
        krotkie_slowa = [s for s in slowa if len(s) <= 3]

        return {
            "slowa_dlugie": len(dlugie_slowa),
            "slowa_krotkie": len(krotkie_slowa),
            "rozne_koncowki": len(set(s[-2:] for s in slowa if len(s) >
2)),
            "powtarzajace_sie": [s for s in set(slowa) if slowa.count(s)
> 1]
        }

    def _analiza_logiczna(self, zdanie: str, slowa: List[str]) -> Dict:
        """Analiza logiczna"""
        spojniki = ["i", "lub", "albo", "czy", "że", "gdy", "ponieważ"]
```

```
        zaprzeczenia = ["nie", "ani", "nigdy", "żaden"]

        return {
            "spojniki": [s for s in slowa if s in spojniki],
            "zaprzeczenia": [s for s in slowa if s in zaprzeczenia],
            "liczba_czynnikow": len([s for s in slowa if
s.endswith(('nie', 'ość', 'stwo'))]),
            "relacje": len([s for s in slowa if s in ["z", "do", "od",
"przez", "dla"]])
        }

    def _analiza_kreatywna(self, zdanie: str, slowa: List[str]) -> Dict:
        """Analiza kreatywna"""
        unikalne_kombinacje = []
        for i in range(len(slowa) - 1):
            kombinacja = f"{slowa[i]}_{slowa[i+1]}"
            unikalne_kombinacje.append(kombinacja)

        return {
            "unikalne_kombinacje": len(set(unikalne_kombinacje)),
            "metafory": len([s for s in slowa if s in ["jak", "niczym",
"podobny", "przypomina"]]),
            "nowe_skojarzenia": random.sample(slowa, min(3, len(slowa)))
if slowa else [],
            "ocena_oryginalnosci": random.uniform(0.3, 0.9)
        }

    def _generuj_wektor(self, analiza: Dict) -> List[float]:
        """Generuje semantyczny wektor na podstawie analizy"""
        wektor = np.zeros(12)  # 12-wymiarowy wektor

        # Wymiary 0-3: Podstawowe statystyki
        wektor[0] = min(analiza.get("dlugosc", 0) / 200, 1.0)
        wektor[1] = min(analiza.get("liczba_slow", 0) / 15, 1.0)
        wektor[2] = analiza.get("unikalne_slowa", 0) /
max(analiza.get("liczba_slow", 1), 1)
        wektor[3] = min(analiza.get("znaki_specjalne", 0) / 5, 1.0)

        # Wymiary 4-7: Specjalizacja neuronu
        if self.type == "analiza":
            wektor[4] = 1.0 if analiza.get("typ_zdania") == "pytanie"
else 0.5
            wektor[5] = min(analiza.get("liczba_czesci", 0) / 3, 1.0)
            wektor[6] = analiza.get("powtorzenia", 0) /
max(analiza.get("liczba_slow", 1), 1)
        elif self.type == "pamiec":
            wektor[4] = min(analiza.get("slowa_dlugie", 0) / 5, 1.0)
            wektor[5] = min(analiza.get("slowa_krotkie", 0) / 5, 1.0)
            wektor[6] = min(analiza.get("rozne_koncowki", 0) / 10, 1.0)
        elif self.type == "logika":
            wektor[4] = min(len(analiza.get("spojniki", [])) / 3, 1.0)
            wektor[5] = min(len(analiza.get("zaprzeczenia", [])) / 2,
1.0)
            wektor[6] = min(analiza.get("liczba_czynnikow", 0) / 5, 1.0)
```

```
        else:  # kreatywnosc
            wektor[4] = min(analiza.get("unikalne_kombinacje", 0) / 10,
1.0)
            wektor[5] = min(analiza.get("metafory", 0) / 2, 1.0)
            wektor[6] = analiza.get("ocena_oryginalnosci", 0.5)

        # Wymiary 8-11: Unikalność i sygnatura
        wektor[7] = random.uniform(0, 0.1)  # Szum
        wektor[8] = hash(zdanie) % 1000 / 1000 if "zdanie" in locals()
else 0.5
        wektor[9] = self.state["aktywnosc"]
        wektor[10] = self.state["doswiadczenie"] / 100
        wektor[11] = sum(wektor[:11]) / 11  # Średnia jako podsumowanie

        return wektor.tolist()

    def _znajdz_podobienstwa(self, nowy_wektor: List[float]) -> Dict:
        """Znajduje podobieństwa z istniejącymi wektorami"""
        if not self.wektory_baza:
            return {"podobne": 0, "najbardziej_podobny": None,
"srednie_podobienstwo": 0}

        podobienstwa = []
        for i, stary_wektor in enumerate(self.wektory_baza):
            # Prosta miara podobieństwa (odległość euklidesowa odwrotnie
proporcjonalna)
            dystans = np.linalg.norm(np.array(nowy_wektor) -
np.array(stary_wektor))
            podobienstwo = 1.0 / (1.0 + dystans)
            podobienstwa.append((i, podobienstwo))

        if podobienstwa:
            najbardziej_podobny = max(podobienstwa, key=lambda x: x[1])
            srednie = sum(p[1] for p in podobienstwa) / len(podobienstwa)

            return {
                "podobne": len([p for p in podobienstwa if p[1] > 0.7]),
                "najbardziej_podobny": najbardziej_podobny,
                "srednie_podobienstwo": srednie
            }

        return {"podobne": 0, "najbardziej_podobny": None,
"srednie_podobienstwo": 0}

    def przetworz_zdanie(self, zdanie: str) -> Dict:
        """Przetwarza zdanie z pełną analizą"""
        start_czas = time.time()

        # Sprawdź długość
        slowa = zdanie.split()
        if not 8 <= len(slowa) <= 12:
            return None

        try:
```

```
            # 1. Analiza semantyczna
            analiza = self._analizuj_zdanie(zdanie)

            # 2. Generuj wektor
            wektor = self._generuj_wektor(analiza)

            # 3. Znajdź podobieństwa
            podobienstwa = self._znajdz_podobienstwa(wektor)

            # 4. Zaktualizuj bazę wektorów
            self.wektory_baza.append(wektor)
            if len(self.wektory_baza) > 100:
                self.wektory_baza.pop(0)

            # 5. Aktualizuj słownik
            for slowo in re.findall(r'\b[a-zA-ZąćęłńóśźżĄĆĘŁŃÓŚŹŻ]+\b',
zdanie.lower()):
                self.slownik[slowo] += 1

            # 6. Oblicz czas
            czas_przetwarzania = time.time() - start_czas

            # 7. Przygotuj wynik
            wynik = {
                "zdanie": zdanie,
                "wektor": wektor,
                "analiza": analiza,
                "podobienstwa": podobienstwa,
                "czas": czas_przetwarzania,
                "slowa": len(slowa),
                "typ": self.type,
                "timestamp": time.time(),
                "neuron_id": self.id,
                "doswiadczenie_neuronu": self.state["doswiadczenie"]
            }

            # 8. Zaktualizuj pamięć
            self.pamiec.append(wynik)

            # 9. Aktualizuj stan neuronu
            self._aktualizuj_stan(czas_przetwarzania, analiza)

            # 10. Znajdź wzorce (co 10 przetworzonych zdań)
            if self.state["przetworzone"] % 10 == 0:
                self._znajdz_wzorce()

            return wynik

        except Exception as e:
            self.statystyki["bledy"] += 1
            print(f"[{self.id}] Błąd przetwarzania: {e}")
            return None

    def _aktualizuj_stan(self, czas: float, analiza: Dict):
```

```
        """Aktualizuje stan neuronu po przetworzeniu"""
        self.state["przetworzone"] += 1
        self.state["doswiadczenie"] = min(100,
self.state["doswiadczenie"] + 0.5)

        # Energia zależna od złożoności
        zlozonosc = analiza.get("unikalne_slowa", 0) /
max(analiza.get("liczba_slow", 1), 1)
        self.state["energia"] = max(0, self.state["energia"] - (0.5 +
zlozonosc))

        # Aktywność rośnie z doświadczeniem
        self.state["aktywnosc"] = min(1.0, self.state["aktywnosc"] +
0.01)

        # Aktualizuj statystyki
        self.statystyki["sredni_czas"] = (
            self.statystyki["sredni_czas"] * (self.state["przetworzone"]
- 1) + czas
        ) / self.state["przetworzone"]

        self.statystyki["unikalne_slowa"] = len(self.slownik)

        # Jeśli energia niska, neuron potrzebuje odpoczynku
        if self.state["energia"] < 20:
            self.state["status"] = "odpoczynek"
            self.state["energia"] = min(100, self.state["energia"] + 30)
        else:
            self.state["status"] = "gotowy"

    def _znajdz_wzorce(self):
        """Automatyczne znajdowanie wzorców w danych"""
        if len(self.wektory_baza) < 10:
            return

        # Prosta analiza klastrów (dla przykładu)
        ostatnie_wektory = self.wektory_baza[-10:]

        # Znajdź podobne grupy
        grupy = []
        for i, v1 in enumerate(ostatnie_wektory):
            grupa = [i]
            for j, v2 in enumerate(ostatnie_wektory):
                if i != j:
                    dystans = np.linalg.norm(np.array(v1) - np.array(v2))
                    if dystans < 0.5:  # Próg podobieństwa
                        grupa.append(j)

            if len(grupa) >= 3 and grupa not in grupy:
                grupy.append(grupa)

        if grupy:
            self.statystyki["wzorce_znalezione"] += len(grupy)
            self.wzorce.append({
```

```
                "timestamp": time.time(),
                "liczba_wzorców": len(grupy),
                "rozmiar_grup": [len(g) for g in grupy]
            })

    def status(self) -> Dict:
        """Zwraca szczegółowy status neuronu"""
        return {
            "id": self.id,
            "typ": self.type,
            "stan": self.state,
            "statystyki": self.statystyki,
            "pamiec": len(self.pamiec),
            "slownik": len(self.slownik),
            "wektory_w_bazie": len(self.wektory_baza),
            "wzorce": len(self.wzorce),
            "specjalizacja": self.state["specjalizacja"]
        }

# === ULEPSZONA KOMÓRKA ===
class KomorkaV7:
    """Ulepszona komórka neuronowa"""

    def __init__(self, komorka_id):
        self.id = komorka_id
        self.neurony = []
        self.kolejka = deque(maxlen=100)
        self.wyniki = []
        self.historia = []
        self.generator = UlepszonyGenerator()

        # Tworzenie 4 zaawansowanych neuronów
        typy = ["analiza", "pamiec", "logika", "kreatywnosc"]
        for i, typ in enumerate(typy):
            neuron_id = f"{komorka_id}-N{i+1}-V7"
            self.neurony.append(NeuronSemantyczny(neuron_id, typ))

        print(f"[{self.id}] Komórka v7.0 utworzona z {len(self.neurony)}
neuronami")

    def dodaj_zdania(self, zdania):
        """Dodaje zdania do kolejki"""
        dodane = 0
        for zdanie in zdania:
            slowa = zdanie.split()
            if 8 <= len(slowa) <= 12:
                self.kolejka.append(zdanie)
                dodane += 1

        self.historia.append({
            "czas": time.time(),
            "akcja": "dodano_zdania",
            "liczba": dodane,
            "kolejka": len(self.kolejka)
```

```
        })

        return dodane

    def przetworz_partie(self, rozmiar=3):
        """Przetwarza partię zdań z inteligentnym wyborem neuronu"""
        wyniki_partii = []

        for _ in range(min(rozmiar, len(self.kolejka))):
            zdanie = self.kolejka.popleft()

            # Inteligentny wybór neuronu
            if "?" in zdanie or "czy" in zdanie.lower():
                neuron = next((n for n in self.neurony if n.type ==
"logika"), self.neurony[0])
            elif len(zdanie) > 100:
                neuron = next((n for n in self.neurony if n.type ==
"analiza"), self.neurony[0])
            elif len(set(zdanie.lower().split())) > 8:
                neuron = next((n for n in self.neurony if n.type ==
"pamiec"), self.neurony[0])
            else:
                # Wybierz neuron z najwyższą energią
                neuron = max(self.neurony, key=lambda n:
n.state["energia"])

            # Przetwórz
            wynik = neuron.przetworz_zdanie(zdanie)
            if wynik:
                wynik["komorka"] = self.id
                wyniki_partii.append(wynik)
                self.wyniki.append(wynik)

        if wyniki_partii:
            self.historia.append({
                "czas": time.time(),
                "akcja": "przetworzono",
                "liczba": len(wyniki_partii),
                "kolejka": len(self.kolejka)
            })

        return wyniki_partii

    def status_komorki(self):
        """Szczegółowy status komórki"""
        statusy_neuronow = [n.status() for n in self.neurony]

        # Statystyki komórki
        lacznie_przetworzone = sum(n.state["przetworzone"] for n in
self.neurony)
        srednia_energia = np.mean([n.state["energia"] for n in
self.neurony])
        laczny_slownik = sum(len(n.slownik) for n in self.neurony)
```

```
        # Efektywność
        if len(self.historia) > 1:
            czas_pracy = self.historia[-1]["czas"] -
self.historia[0]["czas"]
            efektywnosc = lacznie_przetworzone / max(czas_pracy, 1)
        else:
            efektywnosc = 0

        return {
            "id_komorki": self.id,
            "wersja": "7.0",
            "neurony": len(self.neurony),
            "kolejka": len(self.kolejka),
            "przetworzone": len(self.wyniki),
            "statystyki": {
                "lacznie_przetworzone": lacznie_przetworzone,
                "srednia_energia": srednia_energia,
                "laczny_slownik": laczny_slownik,
                "efektywnosc": efektywnosc,
                "historia_akcji": len(self.historia)
            },
            "neurony_status": statusy_neuronow
        }

    def raport_szczegolowy(self):
        """Generuje szczegółowy raport"""
        status = self.status_komorki()

        raport = f"""
        📊 SZCZEGÓŁOWY RAPORT KOMÓRKI {self.id} v7.0
        {'='*50}

        📈 STATYSTYKI OGÓLNE:
        • Neurony: {status['neurony']}
        • W kolejce: {status['kolejka']} zdań
        • Przetworzone: {status['przetworzone']} zdań
        • Łącznie neuronów:
{status['statystyki']['lacznie_przetworzone']}
        • Średnia energia: {status['statystyki']['srednia_energia']:.1f}%
        • Słownik: {status['statystyki']['laczny_slownik']} unikalnych
słów
        • Efektywność: {status['statystyki']['efektywnosc']:.2f}
zdania/sekundę

        □ NEURONY:
        """

        for neuron_status in status["neurony_status"]:
            spec = neuron_status.get("specjalizacja", {})
            spec_str = ", ".join(f"{k}:{v:.2f}" for k, v in spec.items())

            raport += f"""
            {neuron_status['id']} [{neuron_status['typ']:12}]
              Stan: {neuron_status['stan']['status']}
```

```
            Energia: {neuron_status['stan']['energia']:6.1f}%
            Doświadczenie: {neuron_status['stan']['doswiadczenie']:.1f}
            Przetworzone: {neuron_status['stan']['przetworzone']:4d}
            Pamięć: {neuron_status['pamiec']:3d} wpisów
            Słownik: {neuron_status['slownik']:4d} słów
            Wektory: {neuron_status['wektory_w_bazie']:3d}
            Wzorce: {neuron_status['wzorce']:2d}
            Specjalizacja: {spec_str}
        """

    # Ostatnie wyniki
    if self.wyniki:
        raport += f"\n\n📝 OSTATNIE WYNIKI (3):"
        for wynik in self.wyniki[-3:]:
            podobne = wynik.get("podobienstwa", {}).get("podobne", 0)
            raport += f"""
            • {wynik['zdanie'][:60]}...
              Neuron: {wynik['typ']} (podobne: {podobne})
              Czas: {wynik['czas']:.3f}s, Słowa: {wynik['slowa']}
            """

    return raport

# === SYSTEM Z WIZUALIZACJĄ ===
class SystemV7:
    """System v7.0 z wizualizacją i zaawansowanymi funkcjami"""

    def __init__(self):
        self.komorki = []
        self.generator = UlepszonyGenerator()
        self.uruchomiony = False
        self.watek_przetwarzania = None
        self.historia_systemu = []

        print("\n" + "="*60)
        print("□  SYSTEM NEURONOWY v7.0 - ULEPSZONY")
        print("="*60)

    def pokaz_menu(self):
        """Główne menu systemu"""
        while True:
            print("\n" + "="*50)
            print("📋 MENU GŁÓWNE v7.0")
            print("="*50)
            print("1.  Utwórz nową komórkę (v7.0)")
            print("2.  Dodaj zdania do przetwarzania")
            print("3.  Wygeneruj i dodaj zdania")
            print("4.  Uruchom przetwarzanie w tle")
            print("5.  Zatrzymaj przetwarzanie")
            print("6.  Pokaż status wszystkich komórek")
            print("7.  Szczegółowy raport komórki")
            print("8.  Analizuj wyniki")
            print("9.  Eksportuj dane (JSON)")
            print("10. Eksportuj raport (TXT)")
```

```
            print("11. Wyczyść system")
            print("12. Test wydajności")
            print("0.  Wyjdź")
            print("-"*50)

            try:
                wybor = input("Wybierz opcję (0-12): ").strip()

                if wybor == "0":
                    self._wyjscie()
                    break
                elif wybor == "1":
                    self._opcja_1()
                elif wybor == "2":
                    self._opcja_2()
                elif wybor == "3":
                    self._opcja_3()
                elif wybor == "4":
                    self._opcja_4()
                elif wybor == "5":
                    self._opcja_5()
                elif wybor == "6":
                    self._opcja_6()
                elif wybor == "7":
                    self._opcja_7()
                elif wybor == "8":
                    self._opcja_8()
                elif wybor == "9":
                    self._opcja_9()
                elif wybor == "10":
                    self._opcja_10()
                elif wybor == "11":
                    self._opcja_11()
                elif wybor == "12":
                    self._opcja_12()
                else:
                    print("✗ Nieprawidłowy wybór")

            except KeyboardInterrupt:
                print("\n\n□  Przerwano")
                self._wyjscie()
                break
            except Exception as e:
                print(f"✗ Błąd: {e}")

    # OPCJE (podobne ale ulepszone) - pokażę tylko nowe opcje 8, 10, 11,
12

    def _opcja_8(self):
        """8. Analizuj wyniki"""
        if not self.komorki or all(len(k.wyniki) == 0 for k in
self.komorki):
            print("✗ Brak wyników do analizy")
```

```
            return

        print("\n" + "="*50)
        print("📊 ANALIZA WYNIKÓW")
        print("="*50)

        # Zbierz wszystkie wyniki
        wszystkie_wyniki = []
        for komorka in self.komorki:
            wszystkie_wyniki.extend(komorka.wyniki)

        if not wszystkie_wyniki:
            print("⚠️  Brak danych do analizy")
            return

        # Podstawowe statystyki
        print(f"\n📈 STATYSTYKI OGÓLNE:")
        print(f"• Łączna liczba wyników: {len(wszystkie_wyniki)}")
        print(f"• Liczba komórek: {len(self.komorki)}")

        # Średni czas przetwarzania
        czasy = [w.get("czas", 0) for w in wszystkie_wyniki]
        sredni_czas = sum(czasy) / len(czasy)
        print(f"• Średni czas przetwarzania: {sredni_czas:.3f}s")
        print(f"• Najkrótszy czas: {min(czasy):.3f}s")
        print(f"• Najdłuższy czas: {max(czasy):.3f}s")

        # Rozkład typów neuronów
        typy_neuronow = [w.get("typ", "nieznany") for w in
wszystkie_wyniki]
        licznik_typow = {}
        for typ in typy_neuronow:
            licznik_typow[typ] = licznik_typow.get(typ, 0) + 1

        print(f"\n🧠 ROZKŁAD TYPÓW NEURONÓW:")
        for typ, liczba in licznik_typow.items():
            procent = (liczba / len(wszystkie_wyniki)) * 100
            print(f"• {typ:12}: {liczba:3d} ({procent:.1f}%)")

        # Analiza długości zdań
        dlugosci = [w.get("slowa", 0) for w in wszystkie_wyniki]
        print(f"\n📏 DŁUGOŚĆ ZDAŃ:")
        print(f"• Średnia długość: {np.mean(dlugosci):.1f} słów")
        print(f"• Zakres: {min(dlugosci)}-{max(dlugosci)} słów")
        print(f"• Mediana: {np.median(dlugosci):.1f} słów")

        # Unikalne słowa
        wszystkie_slowa = set()
        for wynik in wszystkie_wyniki:
            zdanie = wynik.get("zdanie", "")
            slowa = re.findall(r'\b[a-zA-Ząćęłńóśźż]+\b',
zdanie.lower())
            wszystkie_slowa.update(slowa)
```

```
        print(f"\n📚 SŁOWNICTWO:")
        print(f"• Unikalnych słów: {len(wszystkie_slowa)}")
        print(f"• Przykładowe słowa: {',
'.join(list(wszystkie_slowa)[:10])}")

        # Analiza podobieństw
        if len(wszystkie_wyniki) > 1:
            print(f"\n🔗 ANALIZA PODOBIEŃSTW:")
            liczba_podobnych = sum(
                1 for w in wszystkie_wyniki
                if w.get("podobienstwa", {}).get("podobne", 0) > 0
            )
            print(f"• Zdań z podobieństwami: {liczba_podobnych}")
            print(f"• Procent:
{(liczba_podobnych/len(wszystkie_wyniki))*100:.1f}%")

        # Sugestie
        print(f"\n💡 SUGESTIE:")
        if sredni_czas > 0.1:
            print("• Czas przetwarzania jest wysoki - rozważ
optymalizację")
        if max(czasy) > 0.2:
            print("• Niektóre zdania przetwarzają się bardzo długo")
        if len(wszystkie_slowa) < 50:
            print("• Słownik jest mały - dodaj bardziej różnorodne
zdania")

    def _opcja_10(self):
        """10. Eksportuj raport (TXT)"""
        if not self.komorki:
            print("❌ Brak komórek do raportowania")
            return

        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        nazwa_pliku = f"neuron_raport_{timestamp}.txt"

        with open(nazwa_pliku, 'w', encoding='utf-8') as f:
            f.write("="*70 + "\n")
            f.write("RAPORT SYSTEMU NEURONOWEGO v7.0\n")
            f.write("="*70 + "\n\n")
            f.write(f"Data generowania: {datetime.now().strftime('%Y-%m-
%d %H:%M:%S')}\n")
            f.write(f"Liczba komórek: {len(self.komorki)}\n")
            f.write(f"Łączna liczba neuronów: {len(self.komorki) *
4}\n\n")

            laczne_wyniki = sum(len(k.wyniki) for k in self.komorki)
            f.write(f"Łącznie przetworzonych zdań: {laczne_wyniki}\n")

            for komorka in self.komorki:
                f.write("\n" + "="*60 + "\n")
                f.write(f"KOMÓRKA: {komorka.id}\n")
```

```
                f.write("="*60 + "\n")

                status = komorka.status_komorki()
                f.write(f"\nStatystyki:\n")
                f.write(f"• Neurony: {status['neurony']}\n")
                f.write(f"• W kolejce: {status['kolejka']}\n")
                f.write(f"• Przetworzone: {status['przetworzone']}\n")
                f.write(f"• Średnia energia:
{status['statystyki']['srednia_energia']:.1f}%\n")
                f.write(f"• Słownik:
{status['statystyki']['laczny_slownik']} słów\n")

                f.write(f"\nNeurony:\n")
                for neuron_status in status['neurony_status']:
                    f.write(f"• {neuron_status['id']}: ")
                    f.write(f"E={neuron_status['stan']['energia']:.1f}%,
")
                    f.write(f"P={neuron_status['stan']['przetworzone']},
")
                    f.write(f"S={neuron_status['slownik']} słów\n")

                if komorka.wyniki:
                    f.write(f"\nOstatnie wyniki:\n")
                    for wynik in komorka.wyniki[-3:]:
                        f.write(f"• {wynik['zdanie'][:50]}...\n")
                        f.write(f"  [{wynik['typ']}]
{wynik['czas']:.3f}s, {wynik['slowa']} słów\n")

            f.write("\n" + "="*70 + "\n")
            f.write("KONIEC RAPORTU\n")
            f.write("="*70 + "\n")

        print(f"✅ Raport zapisany do: {nazwa_pliku}")
        print(f"   • Rozmiar: {os.path.getsize(nazwa_pliku) / 1024:.1f}
KB")

    def _opcja_11(self):
        """11. Wyczyść system"""
        if not self.komorki:
            print("ℹ  System jest już pusty")
            return

        potwierdzenie = input("Czy na pewno wyczyścić CAŁY system? (t/n):
").strip().lower()

        if potwierdzenie == 't':
            # Zatrzymaj przetwarzanie jeśli działa
            if self.uruchomiony:
                self.uruchomiony = False
                if self.watek_przetwarzania:
                    self.watek_przetwarzania.join(timeout=1)

            # Podsumowanie przed czyszczeniem
            laczne_wyniki = sum(len(k.wyniki) for k in self.komorki)
```

```
        laczny_slownik = sum(
            sum(len(n.slownik) for n in k.neurony)
            for k in self.komorki
        )

        # Wyczyść
        self.komorki.clear()
        self.historia_systemu.append({
            "czas": time.time(),
            "akcja": "system_wyczyszczony",
            "wyniki_usuniete": laczne_wyniki,
            "slownik_usuniety": laczny_slownik
        })

        print(f"✅ System wyczyszczony")
        print(f"   • Usunięte komórki: {len(self.komorki)}")
        print(f"   • Utracone wyniki: {laczne_wyniki}")
        print(f"   • Utracony słownik: {laczny_slownik} słów")
    else:
        print("❌ Anulowano czyszczenie")

def _opcja_12(self):
    """12. Test wydajności"""
    print("\n" + "="*50)
    print("⚡ TEST WYDAJNOŚCI SYSTEMU")
    print("="*50)

    # Utwórz tymczasową komórkę do testu
    test_komorka = KomorkaV7("TEST-WYDAJNOSCI")

    # Wygeneruj testowe zdania
    print("\n🔧 Przygotowanie testu...")
    test_zdania = self.generator.generuj_partie(20)
    test_komorka.dodaj_zdania(test_zdania)

    # Test 1: Przetwarzanie pojedynczych zdań
    print("\n📊 TEST 1: Przetwarzanie pojedyncze")
    czasy = []
    for zdanie in test_zdania[:5]:
        start = time.time()
        wynik = test_komorka.neurony[0].przetworz_zdanie(zdanie)
        czas = time.time() - start
        czasy.append(czas)
        print(f"   • {zdanie[:40]}...: {czas:.3f}s")

    srednia_poj = sum(czasy) / len(czasy)
    print(f"   ŚREDNIA: {srednia_poj:.3f}s na zdanie")

    # Test 2: Przetwarzanie partii
    print("\n📊 TEST 2: Przetwarzanie partii (5 zdań)")
    start = time.time()
    wyniki = []
    for _ in range(5):
```

```
            if test_komorka.kolejka:
                wynik = test_komorka.przetworz_partie(1)
                if wynik:
                    wyniki.extend(wynik)
        czas_partii = time.time() - start

        print(f"   • Czas całkowity: {czas_partii:.3f}s")
        print(f"   • Średnia: {czas_partii/len(wyniki):.3f}s na zdanie")

        # Test 3: Współbieżność (symulowana)
        print("\n📊 TEST 3: Symulacja współbieżności")
        if len(test_komorka.neurony) >= 4:
            czasy_neuronow = []
            for i, neuron in enumerate(test_komorka.neurony[:4]):
                start = time.time()
                neuron.przetworz_zdanie(test_zdania[i])
                czas = time.time() - start
                czasy_neuronow.append(czas)
                print(f"   • Neuron {neuron.type}: {czas:.3f}s")

            print(f"   • Różnica najszybszy-najwolniejszy:
{max(czasy_neuronow)-min(czasy_neuronow):.3f}s")

        # Podsumowanie
        print("\n" + "="*50)
        print("📈 PODSUMOWANIE WYDAJNOŚCI:")
        print(f"• Pojedyncze zdanie: {srednia_poj:.3f}s")
        print(f"• Partia (5 zdań): {czas_partii:.3f}s
({czas_partii/5:.3f}s na zdanie)")
        print(f"• Zalecana wielkość partii: 3-5 zdań")

        # Usuń tymczasową komórkę
        del test_komorka
        print("\n✅ Test zakończony")

# === URUCHOMIENIE ===
if __name__ == "__main__":
    print("⚡ Ładowanie Systemu Neuronowego v7.0...")
    time.sleep(0.5)

    system = SystemV7()
    system.pokaz_menu()
```

==========
=======
KONIEC PLIKU: neuron_system_v7.py
==========
=======

==========
=======
PLIK 8/14: neuron_systemv6.py

```
======================================================================
=======
Ścieżka:
/storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651/neuron_syste
mv6.py
Rozmiar:    21.9 KB
Rozszerzenie: .py
Linie:      618
Znaki:      21410
Encoding:   utf-8
----------------------------------------------------------------------
-------

# NEURON SWARM SYSTEM v6.0 - MENU SYSTEM
# System neuronowy z menu wyboru opcji

import numpy as np
import time
import random
import threading
from collections import deque
import json
from datetime import datetime
import os
import sys

print("=" * 70)
print("□ SYSTEM NEURONOWY v6.0 - MENU WYBORU")
print("=" * 70)

# === KLASA NEURONU ===
class MenuNeuron:
    def __init__(self, neuron_id, neuron_type):
        self.id = neuron_id
        self.type = neuron_type  # "analiza", "pamiec", "logika",
"kreatywnosc"
        self.state = {
            "aktywnosc": 0.5,
            "energia": 100.0,
            "przetworzone": 0,
            "status": "gotowy"
        }
        self.pamiec = deque(maxlen=20)
        self.slownik = {}

    def przetworz_zdanie(self, zdanie):
        """Przetwarza pojedyncze zdanie"""
        slowa = zdanie.split()
        if not 8 <= len(slowa) <= 12:
            return None

        self.state["przetworzone"] += 1
        self.state["energia"] -= 0.8
```

```
        # Symulacja przetwarzania
        czas = random.uniform(0.05, 0.2)
        time.sleep(czas)

        # Tworzenie wektora (uproszczone)
        wektor = list(np.random.uniform(0, 1, 8))

        # Zapis do pamięci
        wpis = {
            "zdanie": zdanie,
            "wektor": wektor,
            "czas": czas,
            "slowa": len(slowa),
            "typ": self.type,
            "timestamp": time.time()
        }
        self.pamiec.append(wpis)

        # Aktualizacja słownika
        for slowo in slowa:
            self.slownik[slowo.lower()] = self.slownik.get(slowo.lower(),
0) + 1

        return wpis

    def status(self):
        """Zwraca status neuronu"""
        return {
            "id": self.id,
            "typ": self.type,
            "aktywnosc": self.state["aktywnosc"],
            "energia": self.state["energia"],
            "przetworzone": self.state["przetworzone"],
            "pamiec": len(self.pamiec),
            "slownik": len(self.slownik),
            "status": self.state["status"]
        }

# === KLASA KOMÓRKI ===
class MenuKomorka:
    def __init__(self, komorka_id):
        self.id = komorka_id
        self.neurony = []
        self.kolejka = deque(maxlen=50)
        self.wyniki = []
        self.aktywna = True

        # Tworzenie 4 neuronów
        typy = ["analiza", "pamiec", "logika", "kreatywnosc"]
        for i, typ in enumerate(typy):
            neuron_id = f"{komorka_id}-N{i+1}"
            self.neurony.append(MenuNeuron(neuron_id, typ))
```

```
        print(f"[{komorka_id}] Komórka utworzona z {len(self.neurony)}
neuronami")

    def dodaj_zdania(self, zdania):
        """Dodaje zdania do kolejki"""
        dodane = 0
        for zdanie in zdania:
            slowa = zdanie.split()
            if 8 <= len(slowa) <= 12:
                self.kolejka.append(zdanie)
                dodane += 1
        return dodane

    def przetworz_partie(self, rozmiar=3):
        """Przetwarza partię zdań"""
        wyniki_partii = []

        for _ in range(min(rozmiar, len(self.kolejka))):
            if not self.kolejka:
                break

            zdanie = self.kolejka.popleft()

            # Wybierz neuron (rotacyjnie)
            neuron_idx = len(self.wyniki) % len(self.neurony)
            neuron = self.neurony[neuron_idx]

            # Przetwórz
            wynik = neuron.przetworz_zdanie(zdanie)
            if wynik:
                wynik["komorka"] = self.id
                wyniki_partii.append(wynik)
                self.wyniki.append(wynik)

        return wyniki_partii

    def status_komorki(self):
        """Status całej komórki"""
        statusy_neuronow = [n.status() for n in self.neurony]

        return {
            "id_komorki": self.id,
            "neurony": len(self.neurony),
            "kolejka": len(self.kolejka),
            "przetworzone": len(self.wyniki),
            "neurony_status": statusy_neuronow,
            "srednia_energia": np.mean([n.state["energia"] for n in
self.neurony])
        }

    def raport(self):
        """Generuje raport komórki"""
        status = self.status_komorki()
```

```
        raport = f"""
        📊 RAPORT KOMÓRKI {self.id}
        {'='*40}
        • Neurony: {status['neurony']}
        • W kolejce: {status['kolejka']} zdań
        • Przetworzone: {status['przetworzone']} zdań

        🧠 STATUS NEURONÓW:
        """

        for neuron_status in status["neurony_status"]:
            raport += f"""
            {neuron_status['id']} [{neuron_status['typ']:10}]
              Aktywność: {neuron_status['aktywnosc']:.2f}
              Energia: {neuron_status['energia']:6.1f}%
              Przetworzone: {neuron_status['przetworzone']:3d}
              Pamięć: {neuron_status['pamiec']:2d} wpisów
            """

        return raport

# === SYSTEM MENU ===
class SystemMenu:
    def __init__(self):
        self.komorki = []
        self.uruchomiony = False
        self.watek_przetwarzania = None

        # Bank przykładowych zdań
        self.bank_zadan = self._utworz_bank_zadan()

        print("\n" + "="*50)
        print("🧠  SYSTEM NEURONOWY - MENU GŁÓWNE")
        print("="*50)

    def _utworz_bank_zadan(self):
        """Tworzy bank zdań do przetwarzania"""
        szablony = [
            "System {} przetwarza {} w czasie {}.",
            "Neuron {} analizuje {} poprzez {}.",
            "Algorytm {} wykorzystuje {} do {}.",
            "Komórka {} składa się z {} i {}.",
            "Proces {} obejmuje {} oraz {}.",
            "Mechanizm {} umożliwia {} w {}.",
            "Architektura {} zapewnia {} dla {}."
        ]

        podmioty = ["neuronowy", "sztuczny", "biologiczny",
"rozproszony"]
        obiekty = ["dane", "zdania", "wektory", "wzorce", "struktury"]
        przyslowki = ["skutecznie", "szybko", "dokładnie",
"inteligentnie"]

        zdania = []
```

```
        for szablon in szablony:
            zdanie = szablon.format(
                random.choice(podmioty),
                random.choice(obiekty),
                random.choice(przyslowki)
            )

            # Dostosuj długość
            slowa = zdanie.split()
            if len(slowa) < 8:
                zdanie += " " + " ".join(random.sample(obiekty, 8 -
len(slowa)))
            elif len(slowa) > 12:
                zdanie = " ".join(slowa[:12]) + "."

            zdania.append(zdanie.capitalize())

        return zdania

    def _petla_przetwarzania(self):
        """Główna pętla przetwarzania w tle"""
        while self.uruchomiony:
            for komorka in self.komorki:
                if komorka.kolejka:
                    wyniki = komorka.przetworz_partie(2)

                    if wyniki and random.random() < 0.3:
                        print(f"[{komorka.id}] Przetworzono {len(wyniki)}
zdania")

            time.sleep(0.5)

    def pokaz_menu(self):
        """Wyświetla główne menu"""
        while True:
            print("\n" + "="*50)
            print("📋 MENU GŁÓWNE - SYSTEM NEURONOWY")
            print("="*50)
            print("1. Utwórz nową komórkę neuronową")
            print("2. Dodaj zdania do przetwarzania")
            print("3. Uruchom przetwarzanie w tle")
            print("4. Zatrzymaj przetwarzanie")
            print("5. Pokaż status wszystkich komórek")
            print("6. Pokaż szczegółowy raport komórki")
            print("7. Wygeneruj i dodaj nowe zdania")
            print("8. Wyczyść kolejki zdań")
            print("9. Eksportuj wyniki do pliku")
            print("0. Wyjdź z systemu")
            print("-"*50)

            try:
                wybor = input("Wybierz opcję (0-9): ").strip()

                if wybor == "0":
```

```
                self._wyjscie()
                break
            elif wybor == "1":
                self._opcja_1()
            elif wybor == "2":
                self._opcja_2()
            elif wybor == "3":
                self._opcja_3()
            elif wybor == "4":
                self._opcja_4()
            elif wybor == "5":
                self._opcja_5()
            elif wybor == "6":
                self._opcja_6()
            elif wybor == "7":
                self._opcja_7()
            elif wybor == "8":
                self._opcja_8()
            elif wybor == "9":
                self._opcja_9()
            else:
                print("❌ Nieprawidłowy wybór. Spróbuj ponownie.")

        except KeyboardInterrupt:
            print("\n\n□  Przerwano przez użytkownika")
            self._wyjscie()
            break
        except Exception as e:
            print(f"❌ Błąd: {e}")

# === OPCJE MENU ===

def _opcja_1(self):
    """1. Utwórz nową komórkę neuronową"""
    if len(self.komorki) >= 5:
        print("⚠  Osiągnięto maksymalną liczbę komórek (5)")
        return

    if not self.komorki:
        komorka_id = "NEURON-CELL-01"
    else:
        numer = len(self.komorki) + 1
        komorka_id = f"NEURON-CELL-{numer:02d}"

    komorka = MenuKomorka(komorka_id)
    self.komorki.append(komorka)

    print(f"✅ Utworzono komórkę: {komorka_id}")
    print(f"   • Neurony: 4")
    print(f"   • Typy: analiza, pamięć, logika, kreatywność")
    print(f"   • Komórki w systemie: {len(self.komorki)}/5")

def _opcja_2(self):
```

```
        """2. Dodaj zdania do przetwarzania"""
        if not self.komorki:
            print("❌ Najpierw utwórz komórkę (opcja 1)")
            return

        print("\n📝 Wpisz zdania (8-12 słów)")
        print("   Wpisz 'AUTO' dla automatycznych zdań")
        print("   Wpisz 'STOP' aby zakończyć")
        print("-"*40)

        zdania = []
        while True:
            zdanie = input("Zdanie: ").strip()

            if zdanie.upper() == "STOP":
                break
            elif zdanie.upper() == "AUTO":
                # Dodaj 5 automatycznych zdań
                auto_zdania = random.sample(self.bank_zadan, 5)
                zdania.extend(auto_zdania)
                print(f"✅ Dodano 5 automatycznych zdań")
                continue
            elif not zdanie:
                continue

            # Sprawdź długość
            slowa = zdanie.split()
            if not 8 <= len(slowa) <= 12:
                print(f"⚠  Zdanie ma {len(slowa)} słów (wymagane 8-12)")
                kontynuuj = input("Dodać mimo to? (t/n): 
").strip().lower()
                if kontynuuj != 't':
                    continue

            zdania.append(zdanie)

        if not zdania:
            print("⚠  Nie dodano żadnych zdań")
            return

        # Dodaj do wszystkich komórek
        dodane_lacznie = 0
        for komorka in self.komorki:
            dodane = komorka.dodaj_zdania(zdania)
            dodane_lacznie += dodane

        print(f"\n✅ Dodano {dodane_lacznie} zdań do {len(self.komorki)} 
komórek")
        print(f"   • W każdej komórce: ~{dodane_lacznie // 
len(self.komorki) if self.komorki else 0} zdań")

    def _opcja_3(self):
        """3. Uruchom przetwarzanie w tle"""
```

```
        if not self.komorki:
            print("❌ Najpierw utwórz komórkę (opcja 1)")
            return

        if self.uruchomiony:
            print("⚠️  Przetwarzanie jest już uruchomione")
            return

        self.uruchomiony = True
        self.watek_przetwarzania = threading.Thread(
            target=self._petla_przetwarzania,
            daemon=True
        )
        self.watek_przetwarzania.start()

        print("✅ Uruchomiono przetwarzanie w tle")
        print("   • Tryb: ciągły")
        print("   • Komórki: " + ", ".join(k.id for k in self.komorki))
        print("   • Naciśnij opcję 4 aby zatrzymać")

    def _opcja_4(self):
        """4. Zatrzymaj przetwarzanie"""
        if not self.uruchomiony:
            print("⚠️  Przetwarzanie nie jest uruchomione")
            return

        self.uruchomiony = False
        if self.watek_przetwarzania:
            self.watek_przetwarzania.join(timeout=1)

        print("✅ Zatrzymano przetwarzanie")

        # Podsumowanie
        lacznie_wynikow = sum(len(k.wyniki) for k in self.komorki)
        print(f"   • Łącznie przetworzonych: {lacznie_wynikow} zdań")
        print(f"   • Aktywnych komórek: {len(self.komorki)}")

    def _opcja_5(self):
        """5. Pokaż status wszystkich komórek"""
        if not self.komorki:
            print("❌ Brak komórek w systemie")
            return

        print("\n" + "="*60)
        print("📊 STATUS WSZYSTKICH KOMÓREK")
        print("="*60)

        for komorka in self.komorki:
            status = komorka.status_komorki()

            print(f"\n🏢 KOMÓRKA: {status['id_komorki']}")
            print(f"   • Neurony: {status['neurony']}")
```

```
            print(f"   • W kolejce: {status['kolejka']} zdań")
            print(f"   • Przetworzone: {status['przetworzone']} zdań")
            print(f"   • Średnia energia:
{status['srednia_energia']:.1f}%")

            if status['przetworzone'] > 0:
                print(f"   • Ostatnie aktywności:")
                for neuron in status['neurony_status']:
                    if neuron['przetworzone'] > 0:
                        print(f"     {neuron['id']}:
{neuron['przetworzone']} zdań")

        # Statystyki ogólne
        laczna_kolejka = sum(len(k.kolejka) for k in self.komorki)
        laczne_wyniki = sum(len(k.wyniki) for k in self.komorki)

        print(f"\n{'='*60}")
        print("📈 PODSUMOWANIE SYSTEMU:")
        print(f"   • Komórki: {len(self.komorki)}")
        print(f"   • Neurony: {len(self.komorki) * 4}")
        print(f"   • Zdań w kolejkach: {laczna_kolejka}")
        print(f"   • Zdań przetworzonych: {laczne_wyniki}")
        print(f"   • Tryb przetwarzania: {'AKTYWNY' if self.uruchomiony
else 'NIEAKTYWNY'}")
        print(f"{'='*60}")

    def _opcja_6(self):
        """6. Pokaż szczegółowy raport komórki"""
        if not self.komorki:
            print("❌ Brak komórek w systemie")
            return

        print("\n📋 Dostępne komórki:")
        for i, komorka in enumerate(self.komorki, 1):
            status = komorka.status_komorki()
            print(f"   {i}. {komorka.id} "
                  f"(kolejka: {status['kolejka']}, "
                  f"przetworzone: {status['przetworzone']})")

        try:
            wybor = int(input("\nWybierz numer komórki: ")) - 1

            if 0 <= wybor < len(self.komorki):
                komorka = self.komorki[wybor]
                print(komorka.raport())

                # Pokaż ostatnie wyniki
                if komorka.wyniki:
                    print("\n📝 OSTATNIE 3 WYNIKI:")
                    for wynik in komorka.wyniki[-3:]:
                        print(f"\n   Zdanie: {wynik['zdanie'][:60]}...")
                        print(f"   Neuron: {wynik['typ']}")
                        print(f"   Słowa: {wynik['slowa']}")
```

```
                        print(f"   Czas: {wynik['czas']:.3f}s")
                        czas = datetime.fromtimestamp(wynik['timestamp'])
                        print(f"   Godzina: {czas.strftime('%H:%M:%S')}")
            else:
                print("✗ Nieprawidłowy numer komórki")

        except (ValueError, IndexError):
            print("✗ Nieprawidłowy wybór")

    def _opcja_7(self):
        """7. Wygeneruj i dodaj nowe zdania"""
        if not self.komorki:
            print("✗ Najpierw utwórz komórkę (opcja 1)")
            return

        try:
            liczba = int(input("Ile zdań wygenerować? (1-20): "))
            liczba = max(1, min(20, liczba))
        except ValueError:
            liczba = 10
            print(f"⚠  Ustawiono domyślnie: {liczba} zdań")

        # Generuj zdania
        nowe_zdania = []
        for _ in range(liczba):
            szablon = random.choice([
                "System {} przetwarza {} w czasie {}.",
                "Neuron {} analizuje {} poprzez {}.",
                "Algorytm {} wykorzystuje {} do {}."
            ])

            zdanie = szablon.format(
                random.choice(["neuronowy", "sztuczny", "biologiczny"]),
                random.choice(["dane", "zdania", "wektory"]),
                random.choice(["skutecznie", "szybko", "dokładnie"])
            )

            # Dostosuj długość
            slowa = zdanie.split()
            if len(slowa) < 8:
                dodatek = random.sample(["dokładnej", "skutecznej",
"inteligentnej", "automatycznej"],
                                        8 - len(slowa))
                zdanie = zdanie.replace(".", " " + " ".join(dodatek) +
".")

            nowe_zdania.append(zdanie.capitalize())

        # Dodaj do wszystkich komórek
        dodane_lacznie = 0
        for komorka in self.komorki:
            dodane = komorka.dodaj_zdania(nowe_zdania)
            dodane_lacznie += dodane
```

```
        print(f"\n✅ Wygenerowano i dodano {dodane_lacznie} zdań")
        print("   Przykładowe zdania:")
        for i, zdanie in enumerate(nowe_zdania[:3], 1):
            print(f"   {i}. {zdanie}")

    def _opcja_8(self):
        """8. Wyczyść kolejki zdań"""
        if not self.komorki:
            print("❌ Brak komórek w systemie")
            return

        potwierdzenie = input("Czy na pewno wyczyścić wszystkie kolejki?
(t/n): ").strip().lower()

        if potwierdzenie == 't':
            wyczyszczone = 0
            for komorka in self.komorki:
                wyczyszczone += len(komorka.kolejka)
                komorka.kolejka.clear()

            print(f"✅ Wyczyszczono {wyczyszczone} zdań z kolejek")
        else:
            print("❌ Anulowano czyszczenie")

    def _opcja_9(self):
        """9. Eksportuj wyniki do pliku"""
        if not self.komorki:
            print("❌ Brak komórek w systemie")
            return

        laczne_wyniki = sum(len(k.wyniki) for k in self.komorki)
        if laczne_wyniki == 0:
            print("⚠️  Brak wyników do eksportu")
            return

        # Przygotuj dane do eksportu
        dane_eksport = {
            "metadata": {
                "system": "Neuron Swarm System v6.0",
                "timestamp": datetime.now().isoformat(),
                "komorki": len(self.komorki),
                "neurony": len(self.komorki) * 4,
                "laczne_wyniki": laczne_wyniki
            },
            "komorki": []
        }

        for komorka in self.komorki:
            komorka_dane = {
                "id": komorka.id,
                "wyniki": komorka.wyniki[-50:],  # Ostatnie 50 wyników
                "status": komorka.status_komorki()
```

```
            }
            dane_eksport["komorki"].append(komorka_dane)

        # Nazwa pliku
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        nazwa_pliku = f"neuron_wyniki_{timestamp}.json"

        # Zapisz
        try:
            with open(nazwa_pliku, 'w', encoding='utf-8') as f:
                json.dump(dane_eksport, f, ensure_ascii=False, indent=2)

            print(f"✅ Wyniki zapisane do: {nazwa_pliku}")
            print(f"   • Komórki: {len(self.komorki)}")
            print(f"   • Wyniki: {laczne_wyniki}")
            print(f"   • Rozmiar: {os.path.getsize(nazwa_pliku) /
1024:.1f} KB")

        except Exception as e:
            print(f"❌ Błąd zapisu: {e}")

    def _wyjscie(self):
        """0. Wyjdź z systemu"""
        # Zatrzymaj przetwarzanie jeśli działa
        if self.uruchomiony:
            self.uruchomiony = False
            print("⏸  Zatrzymywanie przetwarzania...")
            time.sleep(1)

        # Podsumowanie
        print("\n" + "="*60)
        print("📊 PODSUMOWANIE SESJI")
        print("="*60)

        if self.komorki:
            laczne_wyniki = sum(len(k.wyniki) for k in self.komorki)
            laczna_kolejka = sum(len(k.kolejka) for k in self.komorki)

            print(f"• Utworzone komórki: {len(self.komorki)}")
            print(f"• Łączna liczba neuronów: {len(self.komorki) * 4}")
            print(f"• Przetworzone zdania: {laczne_wyniki}")
            print(f"• Zdania w kolejkach: {laczna_kolejka}")

            if laczne_wyniki > 0:
                print(f"• Średnio na komórkę: {laczne_wyniki /
len(self.komorki):.1f} zdań")

        print("\n👋 Do widzenia!")
        print("="*60)

# === URUCHOMIENIE SYSTEMU ===
if __name__ == "__main__":
    # Sprawdź czy jesteśmy w Spyder
```

```
    try:
        import matplotlib
        print("✅ Środowisko: Spyder IDE")
    except ImportError:
        print("⚠️  Środowisko: Standardowy Python")

    # Inicjalizuj system
    system = SystemMenu()

    # Uruchom menu
    system.pokaz_menu()
```

==========
KONIEC PLIKU: neuron_systemv6.py
==========

==========
PLIK 9/14: newfile.py
==========
Ścieżka: /storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651/newfile.py
Rozmiar: 10.7 KB
Rozszerzenie: .py
Linie: 342
Znaki: 10774
Encoding: utf-8

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
CODE EXPORTER v1.0
Agreguje wszystkie pliki kodowe z katalogu do jednego dokumentu
tekstowego
Przygotowane dla narzędzi AI (Claude, ChatGPT, etc.)

Użycie:
    python code_exporter.py

Wynik:
    code_export.txt - wszystkie kody w jednym pliku
"""

import os
import sys
from datetime import datetime
from pathlib import Path
from typing import List, Dict, Tuple
```

```
#
==========================================================================
===
# KONFIGURACJA
#
==========================================================================
===

# Rozszerzenia plików do eksportu
CODE_EXTENSIONS = {
    # Python
    '.py', '.pyw', '.pyx', '.pyi',
    # Web
    '.html', '.htm', '.css', '.js', '.jsx', '.ts', '.tsx',
    # Data/Config
    '.json', '.yaml', '.yml', '.xml', '.toml', '.ini', '.cfg',
    # Markdown/Docs
    '.md', '.txt', '.rst', '.adoc',
    # Shell/Batch
    '.sh', '.bash', '.bat', '.cmd', '.ps1',
    # C/C++
    '.c', '.cpp', '.h', '.hpp',
    # Other
    '.sql', '.r', '.java', '.go', '.rs', '.swift'
}

# Foldery do ignorowania
IGNORE_DIRS = {
    '__pycache__',
    '.git',
    '.svn',
    '.hg',
    'node_modules',
    'venv',
    'env',
    '.venv',
    '.env',
    'dist',
    'build',
    '.idea',
    '.vscode',
    '.pytest_cache',
    'htmlcov',
    '.tox'
}

# Pliki do ignorowania (wzorce)
IGNORE_FILES = {
    '.gitignore',
    '.dockerignore',
    '.DS_Store',
    'Thumbs.db',
    '.pyc',
```

```
    'code_export.txt',  # Nie eksportuj samego outputu
    'code_exporter.py'  # Opcjonalnie: nie eksportuj samego skryptu
}

# Maksymalny rozmiar pliku (w bajtach) - 1MB
MAX_FILE_SIZE = 1 * 1024 * 1024

# Nazwa pliku wyjściowego
OUTPUT_FILE = "code_export.txt"

#
==========================================================================
===
# FUNKCJE GŁÓWNE
#
==========================================================================
===

def should_ignore_file(filepath: Path) -> bool:
    """Sprawdza czy plik powinien być ignorowany"""
    # Sprawdź nazwę pliku
    if filepath.name in IGNORE_FILES:
        return True

    # Sprawdź wzorce
    for pattern in IGNORE_FILES:
        if pattern.startswith('.') and filepath.name.endswith(pattern):
            return True

    return False


def should_ignore_dir(dirname: str) -> bool:
    """Sprawdza czy folder powinien być ignorowany"""
    return dirname in IGNORE_DIRS or dirname.startswith('.')


def get_file_size_str(size_bytes: int) -> str:
    """Konwertuje rozmiar na czytelny format"""
    for unit in ['B', 'KB', 'MB', 'GB']:
        if size_bytes < 1024.0:
            return f"{size_bytes:.1f} {unit}"
        size_bytes /= 1024.0
    return f"{size_bytes:.1f} TB"


def detect_encoding(filepath: Path) -> str:
    """Wykrywa encoding pliku"""
    encodings = ['utf-8', 'utf-8-sig', 'latin-1', 'cp1252', 'iso-8859-1']

    for encoding in encodings:
        try:
            with open(filepath, 'r', encoding=encoding) as f:
                f.read()
```

```
            return encoding
        except (UnicodeDecodeError, UnicodeError):
            continue

    return 'utf-8'  # Fallback


def scan_directory(root_path: Path) -> List[Path]:
    """
    Skanuje katalog rekurencyjnie i zwraca listę plików do eksportu
    """
    files_to_export = []

    print(f"🔍 Skanowanie katalogu: {root_path}")
    print(f"📁 Ignorowane foldery: {', '.join(sorted(IGNORE_DIRS))}\n")

    for root, dirs, files in os.walk(root_path):
        # Usuń ignorowane foldery z listy (modyfikacja in-place)
        dirs[:] = [d for d in dirs if not should_ignore_dir(d)]

        for filename in files:
            filepath = Path(root) / filename

            # Sprawdź rozszerzenie
            if filepath.suffix.lower() not in CODE_EXTENSIONS:
                continue

            # Sprawdź ignorowane pliki
            if should_ignore_file(filepath):
                continue

            # Sprawdź rozmiar
            try:
                file_size = filepath.stat().st_size
                if file_size > MAX_FILE_SIZE:
                    print(f"⚠  Pominięto (za duży):
{filepath.relative_to(root_path)} ({get_file_size_str(file_size)})")
                    continue
            except OSError:
                continue

            files_to_export.append(filepath)

    return sorted(files_to_export)


def read_file_content(filepath: Path) -> Tuple[str, str]:
    """
    Czyta zawartość pliku
    Zwraca: (content, encoding)
    """
    encoding = detect_encoding(filepath)
```

```
    try:
        with open(filepath, 'r', encoding=encoding, errors='replace') as
f:
            content = f.read()
        return content, encoding
    except Exception as e:
        return f"[BŁĄD ODCZYTU: {e}]", encoding


def create_export_document(files: List[Path], root_path: Path,
output_file: Path):
    """
    Tworzy dokument eksportowy
    """
    print(f"\n📝 Tworzenie dokumentu eksportowego: {output_file}")
    print(f"📊 Liczba plików: {len(files)}\n")

    total_lines = 0
    total_chars = 0
    stats_by_extension = {}

    with open(output_file, 'w', encoding='utf-8') as out:
        # Header
        out.write("=" * 80 + "\n")
        out.write("CODE EXPORT - AGREGACJA KODÓW\n")
        out.write("=" * 80 + "\n\n")
        out.write(f"📅 Data eksportu: {datetime.now().strftime('%Y-%m-%d
%H:%M:%S')}\n")
        out.write(f"📁 Katalog źródłowy: {root_path.absolute()}\n")
        out.write(f"📊 Liczba plików: {len(files)}\n")
        out.write(f"🔧 Rozszerzenia: {',
'.join(sorted(CODE_EXTENSIONS))}\n")
        out.write("\n" + "=" * 80 + "\n\n")

        # Spis treści
        out.write("📋 SPIS TREŚCI\n")
        out.write("-" * 80 + "\n\n")
        for i, filepath in enumerate(files, 1):
            rel_path = filepath.relative_to(root_path)
            out.write(f"{i:3d}. {rel_path}\n")
        out.write("\n" + "=" * 80 + "\n\n")

        # Eksport plików
        for i, filepath in enumerate(files, 1):
            rel_path = filepath.relative_to(root_path)
            file_size = filepath.stat().st_size

            print(f"   {i:3d}/{len(files)} | {rel_path}
({get_file_size_str(file_size)})")

            # Header pliku
            out.write("\n" + "=" * 80 + "\n")
```

```
            out.write(f"PLIK {i}/{len(files)}: {rel_path}\n")
            out.write("=" * 80 + "\n")
            out.write(f"Ścieżka:    {filepath.absolute()}\n")
            out.write(f"Rozmiar:    {get_file_size_str(file_size)}\n")
            out.write(f"Rozszerzenie: {filepath.suffix}\n")

            # Odczyt zawartości
            content, encoding = read_file_content(filepath)
            line_count = content.count('\n') + 1
            char_count = len(content)

            out.write(f"Linie:      {line_count}\n")
            out.write(f"Znaki:      {char_count}\n")
            out.write(f"Encoding:   {encoding}\n")
            out.write("-" * 80 + "\n\n")

            # Zawartość
            out.write(content)

            # Separator
            out.write("\n\n" + "=" * 80 + "\n")
            out.write(f"KONIEC PLIKU: {rel_path}\n")
            out.write("=" * 80 + "\n\n")

            # Statystyki
            total_lines += line_count
            total_chars += char_count
            ext = filepath.suffix.lower()
            if ext not in stats_by_extension:
                stats_by_extension[ext] = {'count': 0, 'lines': 0,
'chars': 0}
            stats_by_extension[ext]['count'] += 1
            stats_by_extension[ext]['lines'] += line_count
            stats_by_extension[ext]['chars'] += char_count

        # Footer - statystyki
        out.write("\n\n" + "=" * 80 + "\n")
        out.write("📊 STATYSTYKI EKSPORTU\n")
        out.write("=" * 80 + "\n\n")
        out.write(f"Łącznie plików:     {len(files)}\n")
        out.write(f"Łącznie linii:      {total_lines:,}\n")
        out.write(f"Łącznie znaków:     {total_chars:,}\n")
        out.write(f"Data eksportu:      {datetime.now().strftime('%Y-%m-
%d %H:%M:%S')}\n")
        out.write("\n")

        # Statystyki per rozszerzenie
        out.write("📈 Statystyki według rozszerzeń:\n")
        out.write("-" * 80 + "\n\n")
        for ext in sorted(stats_by_extension.keys()):
            stats = stats_by_extension[ext]
            out.write(f"{ext:10s} | Plików: {stats['count']:3d} | ")
            out.write(f"Linii: {stats['lines']:6,d} | Znaków:
{stats['chars']:8,d}\n")
```

```
        out.write("\n" + "=" * 80 + "\n")
        out.write("KONIEC DOKUMENTU EKSPORTOWEGO\n")
        out.write("=" * 80 + "\n")

    return total_lines, total_chars, stats_by_extension


def main():
    """Główna funkcja"""
    print("=" * 80)
    print("🚀 CODE EXPORTER v1.0")
    print("=" * 80)
    print()

    # Katalog roboczy
    root_path = Path.cwd()
    output_path = root_path / OUTPUT_FILE

    # Skanuj katalog
    files = scan_directory(root_path)

    if not files:
        print("❌ Nie znaleziono plików do eksportu!")
        print(f"   Sprawdź czy w katalogu są pliki z rozszerzeniami: {',
'.join(sorted(CODE_EXTENSIONS))}")
        return 1

    print(f"✅ Znaleziono {len(files)} plików do eksportu\n")

    # Potwierdzenie
    print("📋 Lista plików do eksportu:")
    for i, filepath in enumerate(files[:10], 1):
        print(f"   {i}. {filepath.relative_to(root_path)}")
    if len(files) > 10:
        print(f"   ... i {len(files) - 10} więcej")

    print()
    response = input("Kontynuować eksport? (t/n): ").strip().lower()
    if response not in ['t', 'tak', 'y', 'yes']:
        print("❌ Anulowano")
        return 1

    # Eksport
    try:
        total_lines, total_chars, stats = create_export_document(files,
root_path, output_path)

        # Podsumowanie
        print("\n" + "=" * 80)
        print("✅ EKSPORT ZAKOŃCZONY POMYŚLNIE!")
        print("=" * 80)
```

```
        print(f"\n📄 Plik wyjściowy: {output_path}")
        print(f"📊 Rozmiar:
{get_file_size_str(output_path.stat().st_size)}")
        print(f"📁 Plików:         {len(files)}")
        print(f"📏 Linii:          {total_lines:,}")
        print(f"🔤 Znaków:         {total_chars:,}")
        print(f"\n💡 Plik gotowy do użycia z narzędziami AI (Claude,
ChatGPT, etc.)")
        print("=" * 80)

        return 0

    except Exception as e:
        print(f"\n❌ BŁĄD podczas eksportu: {e}")
        import traceback
        traceback.print_exc()
        return 1


if __name__ == "__main__":
    try:
        sys.exit(main())
    except KeyboardInterrupt:
        print("\n\n⚠  Przerwano przez użytkownika")
        sys.exit(1)
```

```
================================================================================
KONIEC PLIKU: newfile.py
================================================================================
```

```
================================================================================
PLIK 10/14: orchestrator.py
================================================================================
Ścieżka:
/storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651/orchestrator
.py
Rozmiar:    2.5 KB
Rozszerzenie: .py
Linie:      81
Znaki:      2453
Encoding:   utf-8
--------------------------------------------------------------------------------
```

3

```
# ORCHESTRATOR - Monitorowanie i zarządzanie całym rojem
```

```
import subprocess
import time
import psutil
import json
from tabulate import tabulate

def launch_cell(cell_number, total_cells=10):
    """Uruchom nową komórkę neuronów"""
    # W rzeczywistości uruchamiamy nowy proces Python
    cmd = ["python", "neuron_cell.py"]

    # Symulacja - tutaj rzeczywiste uruchomienie
    print(f"[ORCHESTRATOR] Uruchamiam komórkę #{cell_number}")

    # Zwróć symulowane dane
    return {
        "id": f"CELL-{cell_number:03d}",
        "pid": 1000 + cell_number,
        "start_time": time.time(),
        "neurons": 5,
        "port": 5555 + cell_number
    }

def monitor_swarm(cells):
    """Monitorowanie wszystkich komórek"""
    data = []

    for cell in cells:
        # Symulacja danych (w rzeczywistości pobierane przez socket)
        cpu = psutil.cpu_percent(interval=0.1)
        mem = psutil.virtual_memory().percent

        cell_data = [
            cell["id"],
            cell["pid"],
            "AKTYWNA" if random.random() > 0.1 else "PAUZA",
            cell["neurons"],
            f"{cpu:.1f}%",
            f"{mem:.1f}%",
            time.strftime("%H:%M:%S", time.localtime(cell["start_time"]))
        ]
        data.append(cell_data)

    print("\n" + "="*80)
    print("🎯 ORCHESTRATOR RÓJU NEURONÓW - LIVE MONITOR")
    print("="*80)

    headers = ["ID Komórki", "PID", "Status", "Neurony", "CPU", "RAM",
"Uruchomiona"]
    print(tabulate(data, headers=headers, tablefmt="grid"))

    print(f"\n📈 Podsumowanie: {len(cells)} komórek, {len(cells)*5}
neuronów")
    print("   Użyj Ctrl+C aby zatrzymać monitorowanie")
```

```
if __name__ == "__main__":
    import random

    print("=== SYSTEM RÓJU NEURONÓW - ORCHESTRATOR ===")
    print("1. Uruchom nową komórkę neuronów")
    print("2. Uruchom wiele komórek (klaster)")
    print("3. Monitoruj istniejące komórki")

    choice = input("Wybierz (1-3): ").strip()

    cells = []

    if choice == "2":
        num_cells = int(input("Ile komórek uruchomić? (1-20): "))
        for i in range(num_cells):
            cells.append(launch_cell(i+1, num_cells))
            time.sleep(0.5)  # Opóźnienie między uruchomieniami

    # Monitorowanie
    try:
        while True:
            monitor_swarm(cells)
            time.sleep(5)  # Odśwież co 5 sekund
    except KeyboardInterrupt:
        print("\n\n□ Zatrzymywanie orchestracji...")
```

==========
=======
KONIEC PLIKU: orchestrator.py
==========
=======

==========
=======
PLIK 11/14: requirements.txt
==========
=======
Ścieżka:
/storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651/requirements
.txt
Rozmiar: 204.0 B
Rozszerzenie: .txt
Linie: 11
Znaki: 194
Encoding: utf-8


```
# Neuron Swarm System v7.0
# Minimalne wymagania
numpy>=1.21.0
```

```
# Opcjonalne (do wizualizacji)
# matplotlib>=3.5.0

# Opcjonalne (do zaawansowanej analizy)
# scikit-learn>=1.0.0
# pandas>=1.3.0
```

```
==========================================================================
=======
KONIEC PLIKU: requirements.txt
==========================================================================
=======
```

```
==========================================================================
=======
PLIK 12/14: run.bat
==========================================================================
=======
Ścieżka:
/storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651/run.bat
Rozmiar:    605.0 B
Rozszerzenie: .bat
Linie:      28
Znaki:      560
Encoding:   utf-8
--------------------------------------------------------------------------
-------
```

```
@echo off
echo ========================================
echo □ NEURON SWARM SYSTEM v7.0
echo ========================================
echo.
echo Uruchamianie systemu neuronowego...
echo.

REM Sprawdź czy python jest dostępny
where python >nul 2>nul
if %errorlevel% neq 0 (
    echo ✗ Python nie znaleziony!
    echo Zainstaluj Python 3.8+ z python.org
    pause
    exit /b 1
)

REM Uruchom system
python neuron_system.py

if %errorlevel% neq 0 (
    echo.
    echo ⚠  System zakończył się z błędem
    echo Sprawdź czy wszystkie pliki są w folderze
```

```
)

pause
```

==========
KONIEC PLIKU: run.bat
==========

==========
PLIK 13/14: run_all.bat.txt
==========
Ścieżka: /storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651/run_all.bat.txt
Rozmiar: 177.0 B
Rozszerzenie: .txt
Linie: 5
Znaki: 173
Encoding: utf-8

```
cd /d C:\Users\kamil\roj
start cmd /k "python neuron_cell.py"
start cmd /k "python neuron_cell.py"
start cmd /k "python neuron_cell.py"
start cmd /k "python orchestrator.py"
```

==========
KONIEC PLIKU: run_all.bat.txt
==========

==========
PLIK 14/14: visualizer.py
==========
Ścieżka: /storage/emulated/0/Download/Neuron_Swarm_v7_20251213_102651/visualizer.py
Rozmiar: 4.9 KB
Rozszerzenie: .py
Linie: 115
Znaki: 4860
Encoding: utf-8

```
---------------------------------------------------------------------------
-------

# VISUALIZER - Wizualizacja komunikacji między komórkami

import matplotlib.pyplot as plt
import networkx as nx
import random
from matplotlib.animation import FuncAnimation

class SwarmVisualizer:
    def __init__(self, num_cells=8):
        self.num_cells = num_cells
        self.graph = nx.Graph()

        # Tworzenie komórek
        for i in range(num_cells):
            cell_id = f"C{i+1}"
            self.graph.add_node(cell_id,
                                neurons=5,
                                activity=random.random(),
                                type="cell")

            # Dodaj neurony wewnątrz komórki
            for n in range(5):
                neuron_id = f"{cell_id}-N{n+1}"
                self.graph.add_node(neuron_id,
                                    specialty=["A","M","D","K","L"][n],
                                    type="neuron")
                self.graph.add_edge(cell_id, neuron_id, weight=3)

        # Losowe połączenia między komórkami
        for i in range(num_cells):
            for j in range(i+1, num_cells):
                if random.random() < 0.3:  # 30% szans na połączenie
                    self.graph.add_edge(f"C{i+1}", f"C{j+1}",
                                        weight=random.uniform(0.5, 2),
                                        active=False)

    def update(self, frame):
        """Aktualizacja wizualizacji"""
        plt.clf()

        # Aktualizuj aktywność
        for node in self.graph.nodes():
            if self.graph.nodes[node].get("type") == "cell":
                # Losowa zmiana aktywności
                self.graph.nodes[node]["activity"] = (
                    self.graph.nodes[node]["activity"] * 0.9 +
                    random.random() * 0.1
                )

        # Losowo aktywuj niektóre połączenia
        for u, v in self.graph.edges():
```

```
            if self.graph.nodes[u].get("type") == "cell" and \
               self.graph.nodes[v].get("type") == "cell":
                self.graph[u][v]["active"] = random.random() < 0.2

        # Rysowanie
        pos = nx.spring_layout(self.graph, seed=42)

        # Komórki
        cell_nodes = [n for n in self.graph.nodes()
                     if self.graph.nodes[n].get("type") == "cell"]
        cell_colors = [self.graph.nodes[n]["activity"] for n in
cell_nodes]

        nx.draw_networkx_nodes(self.graph, pos, nodelist=cell_nodes,
                               node_color=cell_colors,
                               node_size=800,
                               cmap=plt.cm.Reds,
                               alpha=0.8)

        # Neurony
        neuron_nodes = [n for n in self.graph.nodes()
                       if self.graph.nodes[n].get("type") == "neuron"]
        nx.draw_networkx_nodes(self.graph, pos, nodelist=neuron_nodes,
                               node_color="lightblue",
                               node_size=200,
                               alpha=0.6)

        # Połączenia wewnętrzne (grube)
        internal_edges = [(u, v) for u, v in self.graph.edges()
                         if self.graph.nodes[u].get("type") !=
self.graph.nodes[v].get("type")]
        nx.draw_networkx_edges(self.graph, pos, edgelist=internal_edges,
                               width=2, alpha=0.5, edge_color="gray")

        # Połączenia zewnętrzne (kolorowe jeśli aktywne)
        external_edges = [(u, v) for u, v in self.graph.edges()
                         if self.graph.nodes[u].get("type") == "cell" and
                         self.graph.nodes[v].get("type") == "cell"]

        active_edges = [e for e in external_edges if
self.graph[e[0]][e[1]].get("active")]
        inactive_edges = [e for e in external_edges if not
self.graph[e[0]][e[1]].get("active")]

        nx.draw_networkx_edges(self.graph, pos, edgelist=active_edges,
                               width=3, alpha=0.8, edge_color="green",
                               style="solid")
        nx.draw_networkx_edges(self.graph, pos, edgelist=inactive_edges,
                               width=1, alpha=0.2, edge_color="gray",
                               style="dashed")

        # Etykiety
        labels = {n: n for n in cell_nodes}
        nx.draw_networkx_labels(self.graph, pos, labels, font_size=10)
```

```
        plt.title(f"Rój Neuronów - {self.num_cells} komórek,
{self.num_cells*5} neuronów\n"
                  f"Frame: {frame}", fontsize=14)
        plt.axis("off")

    def animate(self):
        fig = plt.figure(figsize=(12, 8))
        ani = FuncAnimation(fig, self.update, interval=1000,
cache_frame_data=False)
        plt.show()

if __name__ == "__main__":
    print("🔄 Generowanie wizualizacji roju...")
    visualizer = SwarmVisualizer(num_cells=8)
    visualizer.animate()
```

```
================================================================================
KONIEC PLIKU: visualizer.py
================================================================================
```

```
================================================================================
📊 STATYSTYKI EKSPORTU
================================================================================

Łącznie plików:     14
Łącznie linii:      4,517
Łącznie znaków:     163,934
Data eksportu:      2025-12-13 11:23:10

📈 Statystyki według rozszerzeń:
--------------------------------------------------------------------------------

.bat       | Plików:   1 | Linii:     28 | Znaków:      560
.py        | Plików:  10 | Linii:  4,431 | Znaków:  161,915
.txt       | Plików:   3 | Linii:     58 | Znaków:    1,459

================================================================================
KONIEC DOKUMENTU EKSPORTOWEGO
================================================================================
```